

Complete Course & Laboratory Notes

OBJECT ORIENTED PROGRAMMING WITH JAVA

5 Modules in Full · 11 Lab Programs Compiled & Captured ·
Concept-to-Code Tables · Checkpoint Questions · Viva Bank ·
Rubric-True Record Format

COURSE CODE

1BCS302

Credits	Teaching Hours (L : T : P)	Total Marks	CIE	SEE	Exam Hours	Examination Type
4	3 : 0 : 2 / week	100	50	50	03 Hours	Theory + Lab (IPCC)

by

Dr. Lokesh M R

Professor, Department of Information Science & Engineering
A J Institute of Engineering and Technology (AJIET), Mangaluru, Karnataka, India

Visvesvaraya Technological University (VTU), Belagavi · 2025 Scheme · IPCC
First Edition · September 2026

PUBLICATION PARTICULARS

Course at a Glance

Field	Details
Course	Object Oriented Programming with Java (Integrated Theory & Practical Course — IPCC)
Course Code	1BCS302
Semester	3 (third)
Credits	4
Teaching Hours/Week	3 : 0 : 2 (Lecture : Tutorial : Practical)
Total Hours of Pedagogy	42 (theory) + 28 (lab) + 50 (TW & SL); total hours: 120
Assessment	CIE 50 (theory 25 + practical 25) and SEE 50 · Total 100
Examination type (IPCC)	Theory — CIE + SEE, and Lab — CIE · Exam hours 03
Course Outcomes	CO1 – CO6 (verbatim reproduction in the Syllabus section)
Prescribed textbook	Herbert Schildt and Danny Coward, Java: The Complete Reference, 13th Edition, McGraw-Hill, 2024.

What This Book Covers

Every topic line of the five theory modules is taught the way Java is actually written: the rule, a small program that exercises it, and the exact console output that program produced. Every laboratory item — the ten prescribed experiments and the micro project — is presented complete: the question verbatim, the approach, the full source (including multi-file packages), captured compile-and-run transcripts, observations, result, viva trail and extension exercises.

Part A — Theory Modules 1 to 5: one chapter per module; the module text verbatim, concept tables, and worked demo programs with captured output (twelve programs in all, every one compiled with `javac` and run on a JVM).

Part B — Laboratory Experiments 1 to 11: one chapter per question, shaped as a submission-ready record that mirrors the six university rubric rows; guard runs and second data sets are part of every transcript because the rubrics reward multiple test cases.

Part C — Appendices: a Java quick reference (including the command-line and IDE workflows), the record format with the full assessment rubrics and the 420→15 scaling arithmetic, a consolidated viva bank, and the course resources with the self-study plan.

Provenance of the Code and Output

EVERYTHING PRINTED HERE WAS COMPILED AND EXECUTED

All twenty-three programs in this book (twelve module demos and eleven laboratory solutions, including the micro project) were compiled with `javac` and run on **OpenJDK 11** (openjdk version "11" 2018-09-25, 64-Bit Server VM) on the laboratory Linux machines in September 2026. Every transcript was captured from a real terminal session — a pseudo-terminal, so echoed input lands exactly where a student at the bench would see it; no output was typed by hand or edited afterwards. Where a program deliberately triggers a guard — an empty stack, a zero denominator, an unknown department code, a malformed mark “9x” — that run is printed too, because those transcripts are what the record is graded on. The university note asks for “the most recent version of the IDE tool”; the sources here stick to Java 8-11 constructs so they compile unchanged on any current JDK and in any current IDE (see the environment note in Appendix A).

Imprint

First Edition, September 2026 · Prepared for the VTU 2025 scheme · Course materials follow the university-prescribed syllabus reproduced verbatim in the next section · Errors in printing, if any, are the author's; corrections are welcome through the department office.

FRONT MATTER

Preface

Java is the first language many of these students will meet that makes them name their ideas: a stack is a class, a shape's behaviour is dispatched, a bad input is an exception. The 2025-scheme IPCC structure asks the student to do this in one breath — the theory paper asks why, the laboratory asks show me, and the record is marked against six explicit rubrics. A notes book that separates the two halves serves neither; so this book fuses them.

How Part A works. Each module chapter opens with the syllabus text exactly as prescribed, then teaches every topic line through programs that were actually compiled and run for this book. A typical sequence for “Automatic Type Promotion in Expressions” is: the rule in one sentence, the fragment, the transcript line `byte + byte computed as int = 30`, and the trap that follows (the assignment `byte r = p + q`; is rejected). Concepts are never asserted on the strength of a paragraph; they are asserted on the strength of output the student can reproduce in ten minutes.

How Part B works. The ten experiments and the micro project are written as records you can submit: Aim and objective as the university states them, approach, complete source, the `javac/java` session with two data sets wherever the question allows, observations that reference specific transcript lines, a stated result, viva questions with model answers, and extensions. The six rubric rows of the official CIE table are printed inside each chapter heading block so the student is writing to the marker's sheet from day one; Appendix B reproduces the full rubrics and the arithmetic that turns 420 raw marks into 15.

On the IDE. The syllabus closes its lab note with “make use of the most recent version of the IDE tool for program development” — so this book shows both doors: the plain `javac/java` transcripts (every captured line in the book was produced that way, because a transcript you can reproduce from a shell is worth more than a screenshot) and a worked path through Eclipse/IntelliJ for the same programs, matching the “Tool-Based Learning” method the syllabus itself lists. Nothing in Part B needs an IDE; everything in Part B runs in one.

Three threads tie the parts together. Numbered cross-references: every experiment points back at the module lines it exercises (“Experiment 7 uses `t4_except`”). Honest versioning: all outputs are reproducible on the OpenJDK 11 bench; constructs newer than the syllabus (records, switch expressions, text blocks) are named in the appendices but never silently used. The viva is part of the text: CO6 and the practical-test rubric allocate two marks to communication, so each chapter's answers are written as short spoken sentences, not paragraphs.

FRONT MATTER

Prescribed Syllabus — Verbatim

Reproduced from the VTU 2025-scheme syllabus PDF for 1BCS302 (vtu.ac.in/pdf/2025syll3to8). Table-cell clipping by the PDF extraction is marked [...] — the two affected places are the threading heading in Module 4 and one enumeration method name in Module 5; everything else is word-for-word.

COURSE HEADER · VERBATIM

Course Code 1BCS302 · Scheme 2025 · Type of course IPCC · Semester 3 · Teaching Hours/Week (L:T:P) 3 : 0 : 2 · Total Hours of Pedagogy L:T:P:TW&SL:TH 42 : 0 : 28 : 50 : 120 · Credits 4 · CIE Marks 50 · SEE Marks 50 · Total Marks 100 · Type of Examination (IPCC): Theory — CIE + SEE, and Lab — CIE, Exam Hours 03

COURSE OUTCOME (COURSE SKILL SET) · VERBATIM

At the end of the course, the student will be able to:

1. Demonstrate proficiency in writing simple programs involving branching and looping structures.
2. Construct object-oriented solutions using classes, objects, and methods.
3. Develop programs using inheritance, packages, and interfaces.
4. Make use of exception handling and multithreading in solving complex problems.
5. Apply concepts of autoboxing, enumerations, and generics in program development.
6. Choose a suitable IDE tool to design, develop, and debug Java programs.

MODULE 1 — AN OVERVIEW OF JAVA, DATA TYPES, OPERATORS AND CONTROL STATEMENTS · VERBATIM

An Overview of Java: Object-Oriented Programming (Two Paradigms, Abstraction, The Three OOP Principles), Using Blocks of Code, Lexical Issues (Whitespace, Identifiers, Literals, Comments, Separators, The Java Keywords). Data Types, Variables, and Arrays: The Primitive Types (Integers, Floating-Point Types, Characters, Booleans), Variables, Type Conversion and Casting, Automatic Type Promotion in Expressions, Arrays. Operators: Arithmetic Operators, Relational Operators, Boolean Logical Operators, The Assignment Operator, The ? Operator, Operator Precedence, Using Parentheses. Control Statements: Java's Selection Statements (if, The Traditional switch), Iteration Statements (while, do-while, for, The For-Each Version of the for Loop, Local Variable Type Inference in a for Loop, Nested Loops), Jump Statements (Using break, Using continue, return).

Textbook: Chapters 2, 3, 4, 5 · Number of Hours: 9

MODULE 2 — CLASSES AND METHODS · VERBATIM

Introducing Classes: Class Fundamentals, Declaring Objects, Assigning Object Reference Variables, Introducing Methods, Constructors, the 'this' Keyword, and Garbage Collection. Methods and Classes: Overloading Methods, Using Objects as Parameters, Argument Passing, Returning Objects, Recursion, Introducing Access Control, Understanding static, Introducing final, Introducing Nested and Inner Classes.

Textbook: Chapters 6, 7 · Number of Hours: 8

MODULE 3 — INHERITANCE, PACKAGES AND INTERFACES · VERBATIM

Inheritance: Inheritance Basics, Using super, Creating a Multilevel Hierarchy, When Constructors Are Executed, Method Overriding, Dynamic Method Dispatch, Using Abstract Classes, Using final with Inheritance, Local Variable Type Inference and Inheritance, The Object Class. Packages: Packages, Packages and Member Access, Importing Packages. Interfaces: Interfaces, Default Interface Methods, Use static Methods in an Interface, Private Interface Methods.

Textbook: Chapters 8, 9 · Number of Hours: 9

MODULE 4 — EXCEPTION HANDLING AND MULTITHREADED PROGRAMMING · VERBATIM

Exceptions: Exception-Handling Fundamentals, Exception Types, Uncaught Exceptions, Using try and catch, Multiple catch Clauses, Nested try Statements, throw, throws, finally, Java's Built-in Exceptions, Creating Your Own Exception Subclasses, Chained Exceptions. [...]: Multiple Threads, Using isAlive() and join(), Thread Priorities, Obtaining a Thread's State.

Textbook: Chapters 10, 11 · Number of Hours: 8

MODULE 5 — ENUMERATIONS, AUTOBOXING AND GENERICS · VERBATIM

Enumerations, Type Wrappers and Autoboxing: Enumerations (Enumeration Fundamentals, The values() [...] Methods, Autoboxing/Unboxing Occurs in Expressions, Autoboxing/Unboxing Boolean and Character Values). Generics: A Simple Generics Example, A Generic Class with Two Type Parameters, The General Form of a Generic Class.

Textbook: Chapters 12, 14 · Number of Hours: 8

PRACTICAL COMPONENT OF IPCC · VERBATIM

1. Develop a Java program to add TWO matrices of suitable order N.
2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a Java main method to illustrate Stack operations.
3. Create a Java class called Student with the USN, Name, Branch, Phone details as variables within it. Write a Java program to create N Student objects and print the USN, Name, Branch, and Phone of these objects with suitable headings.
4. Develop a Java program to create a class named Shape. Create three sub classes, namely: circle, triangle and square. Each class has two member functions named draw and erase. Demonstrate the concepts of polymorphism by developing appropriate methods, defining member data, and implementing the main program.
5. Develop a Java program to create an interface Resizable with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.
6. Develop a Java program to create a package named mypack and import & implement it in a suitable class.
7. Develop a Java program to raise a custom exception (user-defined exception) for DivisionByZero using try, catch, throw and finally.
8. Develop a program to create a class MyThread with a class constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.
9. Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using `valueOf()`.
10. Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result.
11. Micro Project: A team of two to three students shall develop a project and submit a brief report for a specific task assigned by the course instructor based on the concepts covered in the syllabus.

Note: Each experiment (1 to 10) will be evaluated for 30 marks (total 300), and the micro-project for 120 marks. Total marks (420) shall be scaled down to 15. Make use of the most recent version of the IDE tool for program development.

FRONT MATTER

Compliance Map — Syllabus to Book

Every prescribed line has a home. Page numbers refer to the body pagination (arabic, printed in the footers); the mapping is generated at build time from the actual pages.

Syllabus item	Content	Pages	CO
MODULE 1	An Overview of Java, Data Types, Operators and Control Statements	2-7	CO1
MODULE 2	Classes and Methods	8-12	CO2
MODULE 3	Inheritance, Packages and Interfaces	13-17	CO3
MODULE 4	Exception Handling and Multithreaded Programming	18-22	CO4
MODULE 5	Enumerations, Autoboxing and Generics	23-27	CO5
PRACTICAL 1	Matrix sum	28-30	CO1
PRACTICAL 2	IntStack (max 10)	31-34	CO2
PRACTICAL 3	Student objects	35-37	CO2
PRACTICAL 4	Shape polymorphism	38-40	CO4
PRACTICAL 5	Resizable interface	41-43	CO3
PRACTICAL 6	mypack package	44-46	CO3
PRACTICAL 7	DivisionByZero exception	47-49	CO4
PRACTICAL 8	MyThread concurrency	50-52	CO4
PRACTICAL 9	Department enum	53-55	CO5
PRACTICAL 10	Wrapper marks stats	56-58	CO5
PRACTICAL 11	Micro project	59--11	CO6
CIE / SEE RULES	Assessment arithmetic and the record rubrics	i-iii (front matter) · App. B	—
TW / SL ACTIVITIES	IDE installation & use (04 h) · Springboard (36 h) · HackerRank (10 h)	App. D	CO6

Assessment Structure (as prescribed)

THEORY CIE · 25	CIE theory component: 25 marks — two Internal Assessment tests, each conducted for 25 marks (first after two modules, second after the remaining three); the average of the two tests scaled down to 25 marks.
PRACTICAL CIE · 25	CIE practical component: 25 marks — 15 marks for the laboratory record via the report rubrics (each experiment evaluated for 30, the sum scaled down to 15) and 10 marks for a 3-hour laboratory test (50 marks scaled down to 10; two internal examiners appointed by the HoD; one experiment excluding the micro project).
ELIGIBILITY	Eligibility: at least 40% of 25, i.e. 10 marks, in the CIE theory component AND at least 40% of 25, i.e. 10 marks, in the CIE practical component.
SEE	SEE: 3-hour theory paper, 50 marks; a minimum of 35%, i.e. 18 marks, is required to pass in the SEE.
PASS	A student successfully completes the course when CIE + SEE totals at least 40 out of 100; below that, the grade is 'F'.

Record-marks arithmetic, verbatim from the practical note: 10 experiments $\times$ 30 = 300, micro project = 120, total 420 $\rightarrow$ scaled down to 15. Each of the six rubric criteria in Appendix B carries 5 marks; $6 \times 5 = 30$ per report.

FRONT MATTER

List of Contents

Page numbers in roman (i-ix) are front matter; the body restarts at arabic 1 and runs continuously through the appendices — both match the printed footers and the PDF bookmarks.

Title Page	i
Course at a Glance	ii
Preface	iii
Prescribed Syllabus — Verbatim	ii
Compliance Maps & Assessment	vi
List of Contents	vii
PART A — THEORY MODULES 1-5	1
Module 1 — An Overview of Java, Data Types, Operators and Control Statements	2
1.1 Concepts in Focus	2
1.2 Worked Demo Programs	3
1.3 Execution (Captured)	5
1.4 Quick-Reference Summary	6
1.5 Checkpoint Questions	7
1.6 Exercises & Practice	7
Module 2 — Classes and Methods	8
2.1 Concepts in Focus	8
2.2 Worked Demo Programs	9
2.3 Execution (Captured)	10
2.4 Quick-Reference Summary	11
2.5 Checkpoint Questions	11
2.6 Exercises & Practice	12
Module 3 — Inheritance, Packages and Interfaces	13
3.1 Concepts in Focus	13
3.2 Worked Demo Programs	14
3.3 Execution (Captured)	16
3.4 Quick-Reference Summary	17
3.5 Checkpoint Questions	17
3.6 Exercises & Practice	17
Module 4 — Exception Handling and Multithreaded Programming	18
4.1 Concepts in Focus	18
4.2 Worked Demo Programs	19
4.3 Execution (Captured)	21
4.4 Quick-Reference Summary	21
4.5 Checkpoint Questions	22
4.6 Exercises & Practice	22
Module 5 — Enumerations, Autoboxing and Generics	23
5.1 Concepts in Focus	23
5.2 Worked Demo Programs	23
5.3 Execution (Captured)	25
5.4 Quick-Reference Summary	26
5.5 Checkpoint Questions	26
5.6 Exercises & Practice	26
PART B — LABORATORY EXPERIMENTS 1-11	27
Experiment 1 — Matrix addition with branching and looping	28
1.1 Approach	28
1.2 Program Code	28

1.3 Compilation & Execution (Captured)	29
1.4 Observations & Analysis	30
1.5 Result	30
1.6 Viva Voce Questions	30
1.7 Exercises & Extensions	30
Experiment 2 — A 10-slot integer stack as a class, driven by a menu	31
2.1 Approach	31
2.2 Program Code	31
2.3 Compilation & Execution (Captured)	32
2.4 Observations & Analysis	33
2.5 Result	33
2.6 Viva Voce Questions	33
2.7 Exercises & Extensions	34
Experiment 3 — N students in an array of objects	35
3.1 Approach	35
3.2 Program Code	35
3.3 Compilation & Execution (Captured)	36
3.4 Observations & Analysis	36
3.5 Result	36
3.6 Viva Voce Questions	36
3.7 Exercises & Extensions	37
Experiment 4 — Shape → circle, triangle, square: polymorphism on stage	38
4.1 Approach	38
4.2 Program Code	38
4.3 Compilation & Execution (Captured)	39
4.4 Observations & Analysis	39
4.5 Result	39
4.6 Viva Voce Questions	39
4.7 Exercises & Extensions	40
Experiment 5 — A Resizable interface implemented by Rectangle	41
5.1 Approach	41
5.2 Program Code	41
5.3 Compilation & Execution (Captured)	42
5.4 Observations & Analysis	42
5.5 Result	42
5.6 Viva Voce Questions	42
5.7 Exercises & Extensions	42
Experiment 6 — A package mypack, imported and used across the boundary	44
6.1 Approach	44
6.2 Program Code	44
6.3 Compilation & Execution (Captured)	45
6.4 Observations & Analysis	45
6.5 Result	45
6.6 Viva Voce Questions	45
6.7 Exercises & Extensions	45
Experiment 7 — A custom DivisionByZeroException with try/catch/throw/throws/finally	47
7.1 Approach	47
7.2 Program Code	47
7.3 Compilation & Execution (Captured)	48
7.4 Observations & Analysis	48
7.5 Result	48
7.6 Viva Voce Questions	48
7.7 Exercises & Extensions	49

Experiment 8 — MyThread: a Thread subclass, concurrent with main	50
8.1 Approach	50
8.2 Program Code	50
8.3 Compilation & Execution (Captured)	51
8.4 Observations & Analysis	51
8.5 Result	51
8.6 Viva Voce Questions	51
8.7 Exercises & Extensions	52
Experiment 9 — Department enum with valueOf() branch lookup	53
9.1 Approach	53
9.2 Program Code	53
9.3 Compilation & Execution (Captured)	54
9.4 Observations & Analysis	54
9.5 Result	54
9.6 Viva Voce Questions	54
9.7 Exercises & Extensions	55
Experiment 10 — Marks as strings → wrapper objects → total & average	56
10.1 Approach	56
10.2 Program Code	56
10.3 Compilation & Execution (Captured)	57
10.4 Observations & Analysis	57
10.5 Result	57
10.6 Viva Voce Questions	57
10.7 Exercises & Extensions	58
Experiment 11 — Micro project — Student Marks Manager	59
11.1 Approach	59
11.2 Program Code	59
11.3 Compilation & Execution (Captured)	61
11.4 Observations & Analysis	62
11.5 Result	62
11.6 Viva Voce Questions	62
11.7 Exercises & Extensions	62
PART C — APPENDICES	63
Appendix A — Java Quick Reference	
Appendix B — Record Format, Rubrics & Assessment	65
Appendix C — Viva Voce Bank	67
Appendix D — Resources, TW/SL & Reproducing	68
About the Author	69

P A R T



Theory — Modules 1 to 5

Every syllabus line of the five modules, taught through programs that were actually compiled and run: each chapter carries the module text verbatim, the concept map with runnable demos, captured javac/java sessions, a quick-reference summary keyed to the prescribed wording, and checkpoint questions in IA-paper shape.

IN THIS PART

- Module 1 — lexical rules, primitives, operators, all four loops
- Module 2 — classes, constructors, this, overloading, static/final, inner
- Module 3 — inheritance and dispatch, packages, interfaces
- Module 4 — try/catch/throw/throws/finally, chaining; threads, join, priorities
- Module 5 — enums, wrappers & autoboxing, generics

Module 1 — An Overview of Java, Data Types, Operators and Control Statements

foundations: overview, types, operators, control

COURSE OUTCOMES	CO1 — branching & looping proficiency
CONTACT & READING	Textbook: Chapters 2, 3, 4, 5 — Number of Hours: 9
DEMOS IN THIS CHAPTER	t1_overview, t1_types, t1_control

AS PRESCRIBED (VERBATIM)

An Overview of Java: Object-Oriented Programming (Two Paradigms, Abstraction, The Three OOP Principles), Using Blocks of Code, Lexical Issues (Whitespace, Identifiers, Literals, Comments, Separators, The Java Keywords). Data Types, Variables, and Arrays: The Primitive Types (Integers, Floating-Point Types, Characters, Booleans), Variables, Type Conversion and Casting, Automatic Type Promotion in Expressions, Arrays. Operators: Arithmetic Operators, Relational Operators, Boolean Logical Operators, The Assignment Operator, The ? Operator, Operator Precedence, Using Parentheses. Control Statements: Java's Selection Statements (if, The Traditional switch), Iteration Statements (while, do-while, for, The For-Each Version of the for Loop, Local Variable Type Inference in a for Loop, Nested Loops), Jump Statements (Using break, Using continue, return).

1.1 Concepts in Focus

Java asks every program to be a class. Before any of the object machinery arrives, the language hands you the procedural toolkit — types, operators, control flow — and this module is that toolkit, plus the lexical rules that decide what the compiler will even accept as spelled.

Two paradigms, one language

A procedural program organises functions that act on data; an object-oriented program organises objects that bundle data with the functions that may touch it. Java supports both, but pushes every line of code inside a class, so even the procedural chapters of this book run inside `class Main`. The syllabus names abstraction as the bridge between the paradigms: the programmer gets to choose what the outside world may see (a `volume()` call) and what it may not (the three doubles behind it). The book's demos keep this concrete — you will watch a private field refuse a caller in Module 2/3, not just read about it.

The three OOP principles named in the syllabus

Principle	What it means in Java	Where this book shows it running
Encapsulation	Fields and the methods that guard them live in one unit; private data with public accessors.	§2.1 Box; Exp. 2's <code>IntStack</code> hides <code>data[]</code> and <code>top</code> .
Inheritance	One class absorbs the members of another and extends them.	§3.1 A→B→C; Exp. 4's Shape family.
Polymorphism	One call site, many behaviours, chosen at run time.	§3.1 <code>a1.speak()</code> ; Exp. 4's <code>Shape[]</code> loop.

Blocks and the lexical level

A pair of braces opens a block: a scope whose variables die at the closing brace (demo `t1_overview` prints a sum computed with a loop variable and a step both born inside inner blocks). At the lexical level Java reads five kinds of token — whitespace (meaningless between tokens, significant inside string literals), identifiers (letters, digits, `_` and `$`, never starting with a digit), literals (decimal, `0x` hex, `0b` binary, `0` octal, underscores allowed for readability), comments (`//`, `/* */`, `/// javadoc`) and separators (`() {} [ ] ; ,` and the operators). The 50 keywords — `class`, `if`, `static`, ... — cannot be identifiers; `true`, `false` and `null` are reserved literals.

Types, casting, promotion

Type	Size	Range (integers)	Note
byte	1 B	−128 ... 127	rarely worth it
short	2 B	−32 768 ... 32 767	rarely worth it

Type	Size	Range (integers)	Note
int	4 B	$-2^{31} \dots 2^{31}-1$	the default for whole numbers
long	8 B	$-2^{63} \dots 2^{63}-1$	suffix L
float	4 B	~7 significant digits	suffix f
double	8 B	~15-16 digits	the default for decimals
char	2 B	0 ... 65 535 (unsigned, Unicode)	note the difference from C
boolean	—	true/false only	0/1 is not a boolean

Widening conversions (byte→short→int→long→float→double) are implicit; narrowing needs a cast, and the cast truncates — the transcript of `t1_types` shows `(int) 9.65 = 9`. Arithmetic smaller than int is promoted before it is computed, which is why `byte + byte` evaluates as int (printed 30) and why `byte r = p + q;` is rejected. The overflow cure is equally small: `Integer.MAX_VALUE + 1` prints -2147483648, while `MAX_VALUE + 1L` prints 2147483648.

Operators and precedence

Arithmetic (+ - * / %), relational (< <= > >= == !=), boolean logic (&&, ||, ! — short-circuit; the bitwise & | ^ work on booleans too), assignment (compound forms += -= *= /= %=) and the ternary `?`, which is Java's only three-operand operator: `(n % 2 == 0 ? "even" : "odd")`. Precedence runs roughly unary → * / % → + - → shifts → relational → equality → logic → ? → assignment; the syllabus's honest advice is printed in its own words as using parentheses whenever the chain grows past two operators.

Control statements

Selection is `if/else` and the traditional `switch` (case labels of integral, char, String or enum types, each terminated with `break`; the syllabus deliberately stops before `switch` expressions). Iteration is `while`, `do-while`, `for` — whose init or update may use Java 10's local-variable type inference, `for (var idx = 0; idx < n; idx++)` — and the `for-each` form `for (int v : marks)`. Jump statements are `break` (leaves the innermost loop or switch), `continue` (next iteration) and `return` (leaves the method, optionally with a value). Demo `t1_control` exercises all of them in one printed page.

1.2 Worked Demo Programs

First program: blocks and the lexical layer

The program below is the book's "hello-class": it demonstrates block scope, the four integer literal bases and the literal families. Read the listing with its output side by side — every literal form named in the syllabus appears exactly once.

src/t1_overview/Main.java · 32 lines, max width 106 cols

```
// Module 1, demo 1 -- anatomy of a first Java program: a class, a main
// entry point, blocks (scope), and the lexical elements of the language:
// whitespace, identifiers, literals, comments and separators.
public class Main {
    public static void main(String[] args) {
        // ---- blocks: every pair of braces opens a new scope -----
        int total = 0;
        {
            // inner block
            int step = 2; // 'step' lives only inside these braces
            for (int i = 1; i <= 5; i = i + step) {
                total += i;
            }
        }
        // step is NOT visible here; using it would be a compile error
        System.out.println("Sum of 1 + 3 + 5      = " + total);

        // ---- integer literals in four bases -----
        int dec = 42, hex = 0x2A, bin = 0b101010, oct = 052;
        System.out.println("42 as decimal/hex/binary/octal = " + dec + "/" + hex + "/" + bin + "/" + oct);

        // ---- floating, long, char, boolean, String literals -----
        long   big = 100_000_000_000L; // underscores for readability
        double flt = 3.14159, sci = 6.02e23;
        char   ch  = 'A', uni = '\u0041'; // unicode escape for 'A'
        boolean ok = true;
    }
}
```

```

        System.out.println("long with underscores    = " + big);
        System.out.println("double + scientific      = " + flt + " / " + sci);
        System.out.println("char 'A' vs \\u0041      = same: " + (ch == uni));
        System.out.println("boolean ok              = " + ok);
        System.out.println("string concat is a separator party: " + "one" + " " + "two");
    }
}

```

Primitives, casting, promotion, arrays

One program measures the type system: wrapper constants print each size and range, a cast truncates, promotion turns byte arithmetic into int, MAX_VALUE overflows, and two array loops do a min/max and a grid.

src/t1_types/Main.java · 41 lines, max width 140 cols

```

// Module 1, demo 2 -- the eight primitive types, conversion and casting,
// automatic type promotion, and arrays (one- and two-dimensional).
public class Main {
    public static void main(String[] args) {
        // ---- size and range of the integer types (wrapper constants) ----
        System.out.println("byte    " + Byte.BYTES + "B [" + Byte.MIN_VALUE + ", " + Byte.MAX_VALUE +
            "]\n");
        System.out.println("short   " + Short.BYTES + "B [" + Short.MIN_VALUE + ", " + Short.MAX_VALUE +
            "]\n");
        System.out.println("int      " + Integer.BYTES + "B [" + Integer.MIN_VALUE + ", " +
            Integer.MAX_VALUE + "]\n");
        System.out.println("long     " + Long.BYTES + "B [" + Long.MIN_VALUE + ", " + Long.MAX_VALUE +
            "]\n");
        System.out.println("double  " + Double.BYTES + "B, float " + Float.BYTES + "B, char " +
            Character.BYTES + "B (boolean: true/false)");

        // ---- widening is implicit; narrowing needs a cast -----
        double x = 9.65;
        int y = (int) x;           // explicit cast drops the fraction
        System.out.println("(int) " + x + " = " + y + " (truncation, not rounding)");

        // ---- automatic type promotion -----
        byte p = 10, q = 20;
        // byte r = p + q;           // compile error: operands promoted to int
        int r = p + q;
        System.out.println("byte + byte computed as int = " + r);
        System.out.println("2 * 3.5 + 1 = " + (2 * 3.5 + 1) + " (int promoted to double)");

        // ---- overflow and the cure -----
        int over = Integer.MAX_VALUE;
        System.out.println("MAX_VALUE + 1 as int = " + (over + 1));
        System.out.println("MAX_VALUE + 1 as long = " + (over + 1L));

        // ---- arrays -----
        int[] marks = {45, 78, 92, 61, 88};
        int max = marks[0], sum = 0;
        for (int v : marks) { sum += v; if (v > max) max = v; }
        System.out.println("marks.length = " + marks.length + ", sum = " + sum + ", max = " + max);

        int[][] grid = new int[3][3];
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                grid[i][j] = (i + 1) * (j + 1);
        System.out.println("multiplication grid row 2: " + grid[1][0] + " " + grid[1][1] + " " + grid[1][
2]);
    }
}

```

Every control statement, one page

The ternary decides odd/even; a char switch selects a multiply; a grade ladder returns from inside a method; while with break/continue counts multiples of 3; a do-while runs once on a false test; for(var ...) sums Fibonacci; a nested loop prints a triangle.

src/t1_control/Main.java · 58 lines, max width 87 cols

```
// Module 1, demo 3 -- control statements: the if ladder, the traditional
// switch, all four loops (while, do-while, for with var, for-each),
// nested loops, break/continue/return and the ? operator.
public class Main {
    static String gradeOf(int marks) {          // demonstrates 'return'
        if (marks >= 90) return "A";
        else if (marks >= 80) return "B";
        else if (marks >= 70) return "C";
        else if (marks >= 60) return "D";
        else return "F";
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println(n + " is " + (n % 2 == 0 ? "even" : "odd"));    // ? operator

        char op = '*';
        double a = 9, b = 3, res;
        switch (op) {              // traditional switch on char
            case '+': res = a + b; break;
            case '-': res = a - b; break;
            case '*': res = a * b; break;
            case '/': res = a / b; break;
            default: res = Double.NaN;
        }
        System.out.println(a + " * " + op + " " + b + " = " + res);

        System.out.print("grade table: ");
        for (int m = 55; m <= 95; m += 10)
            System.out.print(m + "->" + gradeOf(m) + " ");
        System.out.println();

        int i = 0, multiples = 0;
        while (true) {             // break and continue
            i++;
            if (i > 15) break;
            if (i % 3 != 0) continue;
            multiples++;
        }
        System.out.println("multiples of 3 up to 15: " + multiples);

        int k = 10, runs = 0;
        do { runs++; } while (k < 10);    // do always runs once
        System.out.println("do-while runs with false test: " + runs);

        int[] fib = {1, 1, 2, 3, 5, 8};
        int total = 0;
        for (var idx = 0; idx < fib.length; idx++)    // LVTI in a for loop
            total += fib[idx];
        System.out.println("sum of first six Fibonacci numbers: " + total);

        for (var row = 1; row <= 4; row++) {          // nested loops
            for (var col = 1; col <= row; col++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

1.3 Execution (Captured)

Captured terminal sessions — the javac line that built the program and the java run exactly as executed for this book:

First program: blocks and the lexical layer

```
$ javac -d bin/t1_overview src/t1_overview/Main.java
$ java -cp bin/t1_overview Main
Sum of 1 + 3 + 5          = 9
42 as decimal/hex/binary/octal = 42/42/42/42
long with underscores    = 100000000000
```

```
double + scientific      = 3.14159 / 6.02E23
char 'A' vs \u0041      = same: true
boolean ok              = true
string concat is a separator party: one two
```

42 written four ways prints identically four ways: 42/42/42/42 — the base is spelling, not value.

A char literal and its \u0041 escape are the same value (same: true).

step died at its closing brace: the loop variable lives one block deep, the block total lives one level out.

Primitives, casting, promotion, arrays

```
$ javac -d bin/t1_types src/t1_types/Main.java
$ java -cp bin/t1_types Main
byte 1B [-128, 127]
short 2B [-32768, 32767]
int 4B [-2147483648, 2147483647]
long 8B [-9223372036854775808, 9223372036854775807]
double 8B, float 4B, char 2B (boolean: true/false)
(int) 9.65 = 9 (truncation, not rounding)
byte + byte computed as int = 30
2 * 3.5 + 1 = 8.0 (int promoted to double)
MAX_VALUE + 1 as int = -2147483648
MAX_VALUE + 1 as long = 2147483648
marks.length = 5, sum = 364, max = 92
multiplication grid row 2: 2 4 6
```

byte + byte printed as 30 — the sum is an int; assigning it to a byte variable is the compile error to remember.

The overflow pair: MAX_VALUE+1 = -2147483648 but MAX_VALUE+1L = 2147483648 — promotion prevents wraparound.

marks.length = 5, sum = 364, max = 92 — every array carries its own length field.

Every control statement, one page

```
$ javac -d bin/t1_control src/t1_control/Main.java
$ java -cp bin/t1_control Main
7 is odd
9.0 * 3.0 = 27.0
grade table: 55->F 65->D 75->C 85->B 95->A
multiples of 3 up to 15: 5
do-while runs with false test: 1
sum of first six Fibonacci numbers: 20
*
* *
* * *
* * * *
```

Multiples of 3 up to 15: 5 — break at i > 15 and continue on non-multiples did the filtering.

do-while runs with false test: 1 — the test is at the bottom, so one execution is guaranteed.

The Fibonacci sum 20 is computed by a for loop whose counter is declared var — LVTI inferred int from the 0.

1.4 Quick-Reference Summary

Question you may be asked	One-line answer as this book demonstrates it
Why must code live in a class?	Java has no top-level statements; main is a method of a class.
Block vs. body?	Any { ... } opens a scope; a method body is one particular block.
Is (int)9.65 10?	No — casts truncate: 9.
What promotes what?	Anything narrower than int becomes int; mixing with a float/double promotes to that.
switch today?	The traditional switch with case ... break; — that is what the syllabus and both papers ask.
for-each syntax?	for (int v : marks) — no index, no bounds error possible.

1.5 Checkpoint Questions

1. Predict the output: `byte a = 100, b = 100; System.out.println(a * b);`. Now predict `byte c = a * b;`.
2. Why does `t1_control` print `9.0 * 3.0 = 27.0` with a decimal point when the operands were typed 9 and 3?
3. Rewrite the grade ladder with `if/else` only — why can't a switch on `int` marks express "80 and above"?
4. The triangle in the demo prints 4 lines. Change one condition and print 6.
5. Explain what `var` is not: name the misconception the syllabus's phrasing "Local Variable Type Inference" is guarding against.

Model answers to the most-asked questions

Q1. Predict the output: `byte a = 100, b = 100; System.out.println(a * b);`. Now predict `byte c = a * b;`.

Ans. `a * b` prints 10000 — both bytes promote to `int` before multiplying. `byte c = a * b;` is a compile error: an `int` (10 000, and anyway a wider type) cannot narrow into a `byte` without a cast.

Q2. Why does `t1_control` print `9.0 * 3.0 = 27.0` with a decimal point when the operands were typed 9 and 3?

Ans. The operands were declared `double a = 9, b = 3;` widening made them 9.0 and 3.0, so the switch computed in double arithmetic and printed 27.0.

Q3. Rewrite the grade ladder with `if/else` only — why can't a switch on `int` marks express "80 and above"?

Ans. Case labels must be single constant values, not ranges. A switch could map `marks/10` — case 9, 8, 7 ... — which is the standard trick, but the straight-line `if/else` ladder is the honest form for "≥" tests.

Q5. Explain what `var` is not: name the misconception the syllabus's phrasing "Local Variable Type Inference" is guarding against.

Ans. `var` is not "dynamic type". The compiler infers one concrete type from the initializer at compile time (`var idx = 0` is an `int`, forever); it only saves typing. The initializer is therefore mandatory — `var x;` cannot compile.

1.6 Exercises & Practice

1. Extend `t1_overview`: add a literal of every primitive type and print its wrapper's `MAX_VALUE`.
2. Swap the `do-while` test in `t1_control` from `k < 10` to `k > 10`; predict before running.
3. Re-implement the grade ladder with `switch (marks / 10)`; where does the 90-boundary go?
4. Write `min` with `var` over an `int[]` — then try `var m;` and read the error.
5. Convert 2026 by hand: `0b...`? (Use `System.out` to check.)

Module 2 — Classes and Methods

classes and methods in depth

COURSE OUTCOMES	CO2 — classes, objects, methods
CONTACT & READING	Textbook: Chapters 6, 7 — Number of Hours: 8
DEMOS IN THIS CHAPTER	t2_classes, t2_methods

AS PRESCRIBED (VERBATIM)

Introducing Classes: Class Fundamentals, Declaring Objects, Assigning Object Reference Variables, Introducing Methods, Constructors, the ‘this’ Keyword, and Garbage Collection. Methods and Classes: Overloading Methods, Using Objects as Parameters, Argument Passing, Returning Objects, Recursion, Introducing Access Control, Understanding static, Introducing final, Introducing Nested and Inner Classes.

2.1 Concepts in Focus

A class is a template; an object is what new builds from it. Module 2 is the turning point of the course: from here, programs are written as cooperating objects. The demos keep the class small on purpose — a Box with three doubles and a Calc with areas — so the machinery, not the arithmetic, is under the microscope.

Class fundamentals and declaring objects

A class lists fields (state) and methods (behaviour). Declaring `Box b`; only creates a reference; the object appears when `new Box(2.0, 3.0, 4.0)` runs — and `new` does three jobs the syllabus groups under “new Revisited”: allocates memory, runs a constructor, and returns the reference that initialises the variable. Assigning one object variable to another copies the reference, not the object: after `Box d = b`; both names sit on one heap object, so `d.set(10,10,10)` makes `b.volume()` print 1000.0 (transcript t2_classes), and `b == d` is true.

Constructors, this, garbage collection

A constructor has the class’s name, no return type, and runs once per new. A class may declare several — the demo ships a no-arg default that delegates with `this(1, 1, 1)`; a three-double constructor, and a copy constructor `Box(Box b)`. Inside a method, `this` names the receiving object; it is mandatory where parameter names shadow fields (`this.width = w`; inside `set`). When a program drops the last reference to an object (the demo sets `c = null`), the object becomes unreachable; `System.gc()` may suggest a collection cycle, but the JVM keeps the schedule — the guarantee students must state is “memory of unreachable objects is eventually reclaimed”, nothing more.

Overloading and dispatch rules

Overloading = several methods with the same name, distinguished by parameter signature (count, types — never just the return type). `Calc.area` takes one, two, or three doubles; the call picks the arm before run time. `area(3)` widens `int`→`double` and prints 9.0. Overloaded constructors chain with `this(...)` as the first statement.

Objects as parameters, returning objects, recursion

Java passes the value of the reference: a method receiving an object can mutate that object, but re-assigning the parameter inside changes nothing outside. A method may also return a fresh object — the demo’s `enlarged(2.0)` builds and hands back a new Box (volume 192.0 from 24.0). Recursion needs a base case and a stack: `factorial(10)` unfolds to `10·9·...·1` and the transcript prints 3628800.

Access control, static, final, nested and inner

Tool	Rule	Book evidence
Access control	<code>public</code> / <code>protected</code> / <code>(package-private)</code> / <code>private</code> — one file-level <code>private</code> class aside, four levels decide who sees a member.	§3.2’s <code>packageOnly()</code> refuses the outside caller.
<code>static</code>	One member per class, not per object; a static method sees only statics; called as <code>ClassName.m()</code> .	The counter: “static counter after 3 calls: 3”.
<code>final</code>		

Tool	Rule	Book evidence
	final variable = constant; final method = no overriding; final class = no inheriting.	final MAX_STEPS = 100; Module 3 uses final methods.
Nested / inner	A class inside a class; the plain inner one holds the enclosing instance; static nested ones don't.	"inner reads private of outer: 42"; "static nested class here".

2.2 Worked Demo Programs

Box: constructors, this, aliasing, GC reachability

One class, four constructors' worth of lessons: default delegating to the 3-arg one, a copy constructor, setters using this, a method returning a new object, and the moment b and d share one object.

```
src/t2_classes/Main.java · 47 lines, max width 86 cols

// Module 2, demo 1 -- class fundamentals: fields, constructors (default,
// parameterised, overloaded with this(...)), accessors, assigning object
// references (aliasing) and reaching for the garbage collector.
class Box {
    double width, height, depth;

    Box() {                // default constructor
        this(1, 1, 1);      // this(...) -> call the other constructor
    }
    Box(double w, double h, double d) {
        width = w; height = h; depth = d;
    }
    Box(Box b) {           // copy constructor: object as parameter
        width = b.width; height = b.height; depth = b.depth;
    }
    void set(double w, double h, double d) {
        this.width = w; this.height = h; this.depth = d;    // this disambiguates
    }
    double volume() { return width * height * depth; }
    Box enlarged(double f) { // method that returns an object
        return new Box(width * f, height * f, depth * f);
    }
}

public class Main {
    public static void main(String[] args) {
        Box a = new Box();                // 1 x 1 x 1 via this(...)
        Box b = new Box(2.0, 3.0, 4.0);
        Box c = new Box(b);                // copy

        System.out.println("a.volume() = " + a.volume());
        System.out.println("b.volume() = " + b.volume());
        System.out.println("c.volume() = " + c.volume());

        Box d = b;                        // d and b name the SAME object
        d.set(10, 10, 10);
        System.out.println("after d.set(10,10,10), b.volume() = " + b.volume());
        System.out.println("b == d : " + (b == d) + "      (references, not copies)");

        Box e = c.enlarged(2.0);
        System.out.println("c enlarged 2x has volume " + e.volume());

        c = null;                        // last live reference dropped
        System.gc();                      // only a hint -- the JVM decides
        System.out.println("c == null after losing its reference: " + (c == null));
    }
}
```

Calc: overloading, static, final, recursion, inner classes

The deeper half: area() with three signatures, a static call-counter shared by all objects, a final instance constant, a recursive factorial, and both flavours of class-inside-class.

```
src/t2_methods/Main.java · 53 lines, max width 111 cols
```

```
// Module 2, demo 2 -- methods and classes in depth: overloading methods
// and constructors, objects as parameters, returning objects, recursion,
// static members, final fields, nested (static) and inner classes.
class Calc {
    static int calls = 0;           // static field: one copy per class
    int id;                        // instance field
    final int MAX_STEPS = 100;     // final instance constant

    Calc() { this(1); }           // constructor overloading via this
    Calc(int id) { this.id = id; }

    double area(double side)       { calls++; return side * side; }
    double area(double w, double h) { calls++; return w * h; }
    double area(double w, double h, double d) { calls++; return 2 * (w*h + w*d + h*d); }

    static boolean bigger(double[] a, double[] b) { // arrays as parameters
        return a[0]*a[1] > b[0]*b[1];
    }
    long factorial(int n) {        // recursion
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
    static int callsSoFar() { return calls; }      // static method
}

class Outer {
    private int secret = 42;
    class Inner { // member inner class
        int peek() { return secret; } // inner sees private of outer
    }
    static class StaticNested { // nested class, no outer instance
        static String hello() { return "static nested class here"; }
    }
}

public class Main {
    public static void main(String[] args) {
        Calc c = new Calc();
        System.out.println("area(3)      = " + c.area(3));
        System.out.println("area(3, 4)   = " + c.area(3, 4));
        System.out.println("area(3, 4, 5) = " + c.area(3, 4, 5));
        System.out.println("static counter after 3 calls: " + Calc.calls + " (via " + Calc.callsSoFar() +
            ")");
        System.out.println("factorial(10) = " + c.factorial(10));
        System.out.println("final MAX_STEPS = " + c.MAX_STEPS);
        double[] baseA = {5, 6}, baseB = {2, 9};
        System.out.println("5x6 footprint bigger than 2x9? " + Calc.bigger(baseA, baseB));

        Outer o = new Outer();
        Outer.Inner in = o.new Inner();
        System.out.println("inner reads private of outer: " + in.peek());
        System.out.println(Outer.StaticNested.hello());
    }
}
```

2.3 Execution (Captured)

Captured terminal sessions — the javac line that built the program and the java run exactly as executed for this book:

Box: constructors, this, aliasing, GC reachability

```
$ javac -d bin/t2_classes src/t2_classes/Main.java
$ java -cp bin/t2_classes Main
a.volume() = 1.0
b.volume() = 24.0
c.volume() = 24.0
after d.set(10,10,10), b.volume() = 1000.0
b == d : true      (references, not copies)
```

```
c enlarged 2x has volume 192.0
c == null after losing its reference: true
```

a.volume() = 1.0 came from the chain Box() → this(1,1,1) → fields 1,1,1.

b.volume() jumped to 1000.0 after d.set — d is not a copy of b; it is another name on the same object.

c.enlarged(2.0) returned a fresh Box of volume 192.0 while c kept 24.0 — returning objects is routine in Java.

Calc: overloading, static, final, recursion, inner classes

```
$ javac -d bin/t2_methods src/t2_methods/Main.java
$ java -cp bin/t2_methods Main
area(3)          = 9.0
area(3, 4)       = 12.0
area(3, 4, 5)    = 94.0
static counter after 3 calls: 3 (via 3)
factorial(10)    = 3628800
final MAX_STEPS = 100
5x6 footprint bigger than 2x9? true
inner reads private of outer: 42
static nested class here
```

Three different calls bumped one static counter to 3 — static fields belong to the class.

factorial(10) = 3628800: ten stack frames, one base case.

Outer.Inner reached a private field of its Outer instance (42) — Java’s privacy stops outsiders, not its own nested classes.

2.4 Quick-Reference Summary

Term	State it precisely
Object	A runtime instance: state + behaviour, on the heap, named by a reference.
Constructor vs method	Name = class, no return type, runs at new; you never “call” one directly except via this(...)/super(...).
Overload vs override	Overload: same class, different signature, compile time. Override: subclass replaces parent body, run time.
static	Class-level; no this; shared state; call by class name.
Unreachable	No reference path from live variables; eligible for collection; System.gc() is a hint only.

2.5 Checkpoint Questions

1. The demo printed area(3) = 9.0 — why 9.0 and not 9?
2. Sketch the memory picture after Box d = b; d.set(10,10,10);: how many Box objects exist, how many names point at the same one?
3. Write a sameSize(Box o) method that takes an object as a parameter and returns this.volume() == o.volume(). Why must it compare volumes rather than the boxes themselves?
4. What breaks if you add a constructor Box(int w) and call new Box(3) while Box(double ...) arms exist?
5. Give one reason each to make a member static, final, and inner.

Model answers to the most-asked questions

Q1. The demo printed area(3) = 9.0 — why 9.0 and not 9?

Ans. Overloads matched area(double); the int 3 widened to 3.0 before the call, and a double prints with a decimal point.

Q3. Write a sameSize(Box o) method that takes an object as a parameter and returns this.volume() == o.volume(). Why must it compare volumes rather than the boxes themselves?

Ans. Because b == o compares references (identity); two distinct boxes can share a volume. The value comparison must be on the numbers, exactly as the equals-style reasoning demands.

Q4. What breaks if you add a constructor Box(int w) and call new Box(3) while Box(double ...) arms exist?

Ans. Nothing breaks — int widens to double, so new Box(3) silently selects the double arm. That silent widening is the reason overloaded numeric constructors need comments; the classic bug hunt in viva.

Q5. Give one reason each to make a member static, final, and inner.

Ans. static for data the class owns once (a counter); final to pin a constant per object or to seal a method; inner when an object genuinely needs its enclosing instance — the 42-peeking Inner in the demo is exactly that case.

2.6 Exercises & Practice

1. Add a `Box(Box src, double grow)` constructor; make `enlarged(2.0)` use it.
2. Make the calls counter print per-object counts too — which of the two is static, and why can `bigger()` be static?
3. Overload factorial for `BigInteger`? Explain why the compiler refuses to pick between `f(long)` and `f(int)` for `factorial(10)` once both exist.
4. Break `t2_classes` on purpose: call `c.width` after `c = null;;` capture the exact error text for your record.
5. Turn `StaticNested` into an inner class and call `hello()` without an `Outer` instance — document the error.

Module 3 — Inheritance, Packages and Interfaces

inheritance, packages, interfaces

COURSE OUTCOMES	CO3 — inheritance, packages, interfaces
CONTACT & READING	Textbook: Chapters 8, 9 — Number of Hours: 9
DEMOS IN THIS CHAPTER	t3_inherit, t3_pkg, t3_iface

AS PRESCRIBED (VERBATIM)

Inheritance: Inheritance Basics, Using super, Creating a Multilevel Hierarchy, When Constructors Are Executed, Method Overriding, Dynamic Method Dispatch, Using Abstract Classes, Using final with Inheritance, Local Variable Type Inference and Inheritance, The Object Class. Packages: Packages, Packages and Member Access, Importing Packages. Interfaces: Interfaces, Default Interface Methods, Use static Methods in an Interface, Private Interface Methods.

3.1 Concepts in Focus

Inheritance is Java's reuse engine; packages and interfaces are its organising plates. The module is best read as one sentence: the reference type decides what you may call, the object decides what happens. Every demo below proves one half of that sentence.

Inheritance basics and constructor order

class B extends A copies A's accessible members into B's namespace. Building a C in an A→B→C chain prints the constructors base-first — A(), B(), C() in t3_inherit — because “when constructors are executed” has one answer: each constructor implicitly (or explicitly, via super(name)) runs its parent's first. A child with its own constructor that needs parent arguments must call super(args) as its first line — Bank/Student/Shape in the experiments all do it.

Overriding, dispatch, abstract

Overriding gives the subclass its own body for the same signature; the parent's void speak() printed “Thing makes a generic sound”, the child's printed “Rex barks” — and both went through the same call text a1.speak(); on an Animal reference. That is dynamic method dispatch: the JVM looks up the object's class at run time. An abstract class may hold concrete methods but declares some without a body; you cannot instantiate it, and every abstract method must be overridden — Figure2's area() forced Rect2 to supply one, and through the parent reference the demo printed “a figure with area 12.0”. final meets inheritance twice: a final method cannot be overridden (Dog.fetch), a final class cannot be extended at all.

LVTI and inheritance; the Object class

var and inheritance coexist quietly: var d = new Dog("Rex"); infers the exact class Dog — losing nothing — but you can never write var x; or re-assign a parent object into a Dog-typed var, because its type is fixed at first assignment. Every class without an extends silently extends Object, which supplies toString(), equals(), hashCode(), getClass(), notify()/wait() and the thread hooks; the demo's “inherited toString(): Dog[Rex]” line is an overridden Object method, and “same ref → true, distinct objects → false” is equals() still behaving like == until someone overrides it.

Packages, member access, importing

A package is a namespace: the first line of every file, package mypack;, and a matching directory layout. Without an import, callers must write the fully qualified name (new mypack.ToolKit() in t3_pkg does exactly that once); with it, the short name works. Packages and member access are the same lesson: public crosses package borders, the package-private (no-modifier) level does not — packageOnly() stayed inside mypack and had to be re-exported through a bridge class to print its 99.

Interfaces: default, static, private

An interface is a contract of abstract methods a class promises to implement — Java's way to have behaviour without inheriting state. A default method carries a real body and is inherited unless overridden (Movable.report() decorated both carts). A static interface method is a utility attached to the type — the demo used it as a factory (Movable.makeDefault()). A private interface method (Java 9+) exists so two de-

faults can share helper code without exposing it — `decorate()` was never callable from the implementing class. Interface variables are implicitly public static final: `BASE = 1.0`.

3.2 Worked Demo Programs

Constructor order, dispatch, abstract, Object

Six classes in one file trace the whole mechanism: multilevel prints in base-first order; `Animal/Dog` override `speak()`; `Figure2` forces `area()`; `toString` and `equals` come from `Object`.

```
src/t3_inherit/Main.java · 46 lines, max width 117 cols

// Module 3, demo 1 -- inheritance: base-first constructor execution in a
// multilevel hierarchy, explicit super(...), method overriding, dynamic
// method dispatch, abstract classes, final methods and the Object class.
class A { A() { System.out.println(" A() constructor"); } }
class B extends A { B() { System.out.println(" B() constructor"); } }
class C extends B { C() { System.out.println(" C() constructor"); } }

class Animal {
    protected String name;
    Animal(String name) { this.name = name; }
    void speak() { System.out.println(name + " makes a generic sound"); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + name + "]"; }
}
class Dog extends Animal {
    Dog(String name) { super(name); } // explicit super
    @Override void speak() { System.out.println(name + " barks"); } // override
    final void fetch() { System.out.println(name + " fetches"); } // final method
}

abstract class Figure2 { // abstract class demo
    abstract double area(); // must be supplied by subclasses
    String describe() { return "a figure with area " + area(); } // concrete part
}
class Rect2 extends Figure2 {
    double w, h;
    Rect2(double w, double h) { this.w = w; this.h = h; }
    double area() { return w * h; }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("multilevel A -> B ->C (base first, derived last):");
        C c = new C();

        Animal a1 = new Dog("Rex"); // parent reference to child object
        Animal a2 = new Animal("Thing");
        a1.speak(); // dispatch decides at RUNTIME
        a2.speak();
        System.out.println("inherited toString(): " + a1);
        System.out.println("Object.equals: same ref -> " + a1.equals(a1) + ", distinct objects -> " +
a1.equals(a2));

        Figure2 f = new Rect2(3, 4);
        System.out.println("through the abstract type: " + f.describe());
        ((Dog) a1).fetch(); // downcast to reach a Dog-only method
    }
}
```

mypack: three classes, one bridge, one import

`Calc` lives in `mypack` with public arithmetic methods and one package-private helper; `ToolKit`, also inside `mypack`, re-exports the helper; the default-package `Main` imports `Calc` and fully qualifies `ToolKit`.

```
src/t3_pkg/Main.java · 13 lines, max width 116 cols

import mypack.Calc; // import a public class from the package
// The default package cannot see mypack's package-private members --
// ToolKit (a mypack citizen) acts as the bridge.
public class Main {
    public static void main(String[] args) {
```

```

    Calc calc = new Calc();
    System.out.println("imported mypack.Calc into the default package");
    System.out.println("add(12, 30)      = " + calc.add(12, 30));
    System.out.println("subtract(50, 8)  = " + calc.subtract(50, 8));
    System.out.println("multiply(7, 6)   = " + calc.multiply(7, 6));
    System.out.println("packageOnly() seen from the bridge class: " + new
mypack.ToolKit().usePackagePrivate());
    }
}

```

src/t3_pkg/mypack/Calc.java · 9 lines, max width 85 cols

```

package mypack;
// Module 3, demo 2 -- a class that lives in the package mypack.
public class Calc {
    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }

    int packageOnly() { return 99; } // package-private: NOT visible outside mypack
}

```

src/t3_pkg/mypack/ToolKit.java · 5 lines, max width 71 cols

```

package mypack;
// Sits INSIDE mypack, so it may legally touch packageOnly().
public class Toolkit {
    public int usePackagePrivate() { return new Calc().packageOnly(); }
}

```

Movable: default, static and private methods together

An interface with two abstract methods, one default, one private helper, one static factory and one constant; Cart implements only the two abstracts.

src/t3_iface/Main.java · 40 lines, max width 105 cols

```

// Module 3, demo 3 -- interfaces: abstract methods, constants, a default
// method, a static interface method (factory) and a private interface
// method that the default method calls (Java 9 onwards).
interface Movable {
    double BASE = 1.0; // implicitly public static final

    void moveBy(double d); // implicitly public abstract
    double last();

    default String report() { // default method with a body
        return decorate("moved by " + (last() * BASE));
    }
    private String decorate(String s) { // private interface method (Java 9+)
        return "<" + s + ">";
    }
    static Movable makeDefault() { // static interface method
        return new Cart(0);
    }
}

class Cart implements Movable {
    private double pos;
    Cart(double pos) { this.pos = pos; }
    public void moveBy(double d) { pos += d; }
    public double last() { return pos; }
}

public class Main {
    public static void main(String[] args) {
        Movable m = new Cart(10); // interface reference to impl object
        m.moveBy(4);
        System.out.println("after moveBy(4): " + m.report());
        Movable d = Movable.makeDefault();
        d.moveBy(2.5 * Movable.BASE);
    }
}

```

```

        System.out.println("factory-built cart: " + d.report());
        System.out.println("interface constant BASE = " + Movable.BASE);
        // The variable 'm' cannot see Cart's constructor-only members:
        System.out.println("implementation class behind the interface: " + m.getClass().getSimpleName());
    }
}

```

3.3 Execution (Captured)

Captured terminal sessions — the javac line that built the program and the java run exactly as executed for this book:

Constructor order, dispatch, abstract, Object

```

$ javac -d bin/t3_inherit src/t3_inherit/Main.java
$ java -cp bin/t3_inherit Main
multilevel A -> B ->C (base first, derived last):
  A() constructor
  B() constructor
  C() constructor
Rex barks
Thing makes a generic sound
inherited toString(): Dog[Rex]
Object.equals: same ref -> true, distinct objects -> false
through the abstract type: a figure with area 12.0
Rex fetches

```

The three constructor lines printed A(), B(), C() — parent construction is not optional, it is the first statement of every child constructor.

a1.speak() with a1 typed Animal printed “Rex barks”: dispatch chose Dog’s body at run time.

((Dog) a1).fetch() needed a cast — the parent handle cannot see child-only methods; that is the price of programming to the supertype.

mypack: three classes, one bridge, one import

```

$ javac -d bin/t3_pkg src/t3_pkg/Main.java src/t3_pkg/mypack/Calc.java src/t3_pkg/mypack/ToolKit.java
$ java -cp bin/t3_pkg Main
imported mypack.Calc into the default package
add(12, 30)      = 42
subtract(50, 8)  = 42
multiply(7, 6)   = 42
packageOnly() seen from the bridge class: 99

```

All three public calls returned the same value 42 — 12+30, 50–8 and 7×6 were chosen so a mis-wired switch shows itself.

packageOnly() seen only through the bridge: 99 — the import brought public members, not the package’s private furniture.

One file per public class, one directory per package: javac received three sources, in order, and produced three .class files inside bin/t3_pkg/mypack/... as the transcript header shows.

Movable: default, static and private methods together

```

$ javac -d bin/t3_iface src/t3_iface/Main.java
$ java -cp bin/t3_iface Main
after moveBy(4): <moved by 14.0>
factory-built cart: <moved by 2.5>
interface constant BASE = 1.0
implementation class behind the interface: Cart

```

Both prints ran through the interface variable: m.report() and d.report() executed the interface’s own default method.

makeDefault() built a Cart without the caller ever naming the class — static interface methods make the type a factory.

decorate() never appears in main’s vocabulary: private interface methods stay invisible to implementors.

3.4 Quick-Reference Summary

Rule	Say it exactly
super	First statement of the child constructor (or implicit call to the parent's no-arg form); <code>super.field</code> disambiguates hidden names.
Override	Same signature, same-or-wider access, run-time replaces parent behaviour; <code>@Override</code> lets the compiler check you meant it.
Dispatch	Method chosen by the object's class, not the reference's type.
abstract	Class can't be instantiated; each abstract method must be implemented somewhere down the chain.
Package	Directory-shaped namespace; public crosses, package-private stays.
Interface	Contract of abstract methods + default/static/private machinery; a class implements any number.

3.5 Checkpoint Questions

1. Why does the transcript show exactly one `B()` line and not two when `C` is constructed?
2. Write the one-line declaration that would make `Rect2` fail to compile, and name the rule it breaks.
3. In `t3_pkg`, `Main` calls `new Calc().packageOnly()` directly. Predict the compiler's verdict; then check against the source comment.
4. An interface variable `Movable m = new Cart(10);` — list what `m` may call and what is hidden. Relate it to Exp. 5's `Resizable`.
5. default methods vs. abstract methods: when does adding a method to an interface not break every implementor?

Model answers to the most-asked questions

Q1. Why does the transcript show exactly one `B()` line and not two when `C` is constructed?

Ans. `C`'s constructor runs `B`'s once, and `B`'s runs `A`'s once — one chain, one visit per class. `B()` prints from inside `B`'s constructor, which the chain invokes a single time.

Q2. Write the one-line declaration that would make `Rect2` fail to compile, and name the rule it breaks.

Ans. Deleting `double area() { ... }` from `Rect2`: an abstract method left unimplemented in a concrete class is a compile error.

Q4. An interface variable `Movable m = new Cart(10);` — list what `m` may call and what is hidden. Relate it to Exp. 5's `Resizable`.

Ans. `m` may call `moveBy`, last, the inherited `report`, and (via the type) `BASE`; `Cart`-only members — its constructor, any extra methods — are invisible until a cast. Exp. 5 drives `resizeWidth/resizeHeight` through the `Resizable` handle for exactly this reason.

Q5. default methods vs. abstract methods: when does adding a method to an interface not break every implementor?

Ans. When the new method is default: the interface supplies a body, implementors inherit it untouched — the mechanism behind Java's own library evolution (e.g. `Collection.stream`).

3.6 Exercises & Practice

1. Insert a fourth level `D` in the hierarchy; predict then verify the constructor print order.
2. Override `speak()` in `Dog` and also overload `speak(int volume)` in `Animal` — show one call each.
3. Give `Figure2` a constructor and make `Rect2` call `super(...)`; when must that `super` line appear?
4. Move `ToolKit` out of `mypack`; watch `packageOnly()` disappear from its reach — explain why.
5. Add default `boolean isMoving() { return last() != 0; }` to `Movable` and call it from `Main`.

Module 4 — Exception Handling and Multithreaded Programming

exceptions and threads

COURSE OUTCOMES	CO4 — exception handling, multithreading
CONTACT & READING	Textbook: Chapters 10, 11 — Number of Hours: 8
DEMOS IN THIS CHAPTER	t4_except, t4_threads

AS PRESCRIBED (VERBATIM)

Exceptions: Exception-Handling Fundamentals, Exception Types, Uncaught Exceptions, Using try and catch, Multiple catch Clauses, Nested try Statements, throw, throws, finally, Java's Built-in Exceptions, Creating Your Own Exception Subclasses, Chained Exceptions. [...]: Multiple Threads, Using isAlive() and join(), Thread Priorities, Obtaining a Thread's State.

4.1 Concepts in Focus

Two failure-shaped topics. Exceptions decide what happens when the normal path breaks; threads decide how several normal paths coexist. Both are graded in the lab papers as much for the conversation (why this catch, why that join) as for the code.

Exception fundamentals

When a statement fails, the runtime throws an object describing the failure; the stack unwinds until someone catches it. Uncaught, it reaches the thread's top, prints a stack trace and kills the thread — that is the “uncaught exception” the syllabus names before teaching the cure. All exceptions descend from Throwable; the book keeps the taxonomy exam-shaped: Error (OutOfMemoryError — don't catch), Exception and inside it the unchecked RuntimeException family (ArithmeticException, NumberFormatException, NullPointerException) versus the checked remainder (IOException, SQLException) which the compiler forces you to face with try/catch or a throws clause.

try, catch, multiple/nested, throw, throws, finally

try { risky; } catch (X e) { rescue; } finally { cleanup; } — the try guards, the catch handles one declared type (or a supertype), finally runs always: the safeDivide transcript shows “finally always runs” on both the healthy line ($10/2 = 5$) and the wreck. Multiple catches order from specific to general — NumberFormatException outside a narrower ArithmeticException inside nested try caught “abc” and the divide-by-zero input 0 on their own lines. throw raises an instance; throws declares one. The difference is one line of grammar that viva asks in ten flavours.

Own subclasses and chaining

A custom exception extends Exception (checked) or RuntimeException (unchecked), usually with the two standard constructors. The book's LowBalanceException also accepts a Throwable cause and forwards it to super(msg, cause) — that is a chained exception, and the transcript shows the payoff: caught prints “insufficient funds”, getCause() prints the wrapped “java.io.IOException: ledger read failed at record 1000” underneath.

Threads: two paths in one program

A thread is an independent path of execution; the JVM runs main as one and lets you add others. Two ways to define a worker: extend Thread (Module 4 demo; the experiment form) or implement Runnable and hand it to a Thread — same effect, different inheritance budget. start() asks the scheduler to enter run(); calling run() directly just runs a method on the caller. isAlive() answers between start and death; join() makes the caller wait for that thread to end; getState() exposes the lifecycle (NEW → RUNNABLE ⇌ TIMED_WAITING → TERMINATED, as the transcript caught them). Priorities are hints — setPriority(Thread.MAX_PRIORITY) (10) against MIN_PRIORITY (1) changed the book's demo numbers, not its order; the syllabus line says “thread priorities”, the honest sentence about them is “the scheduler decides”.

REPRODUCIBILITY NOTE

Interleaving is real, which makes naive multi-thread output non-deterministic — and a lab record whose transcript changes every run is a viva liability. Both the demo and Experiment 8 therefore buffer each child's lines and flush after `join()`, so the printed record is stable while the concurrency remains genuine (main's own ticks interleave with the children's sleeps).

4.2 Worked Demo Programs

One program, ten exception verbs

ArithmeticException caught in a method whose finally prints twice; a nested try separates number-vs-zero diagnosis; a checked LowBalanceException with an IOException cause is caught, unwrapped, and the balance read after recovery.

src/t4_except/Main.java · 67 lines, max width 93 cols

```
// Module 4, demo 1 -- exception handling: built-in types, try/catch with
// multiple and nested catches, throw, throws, finally, a custom subclass
// of Exception, and a chained exception (with a cause).
import java.io.IOException;

class LowBalanceException extends Exception {          // a checked exception of our own
    LowBalanceException(String msg)                    { super(msg); }
    LowBalanceException(String msg, Throwable cause) { super(msg, cause); } // chained
}

class Bank {
    private int balance = 100;
    void withdraw(int amount) throws LowBalanceException {
        if (amount <= 0)
            throw new IllegalArgumentException("non-positive amount"); // throw
        if (amount > balance)
            throw new LowBalanceException("insufficient funds", // wraps a cause
                new IOException("ledger read failed at record " + amount));
        balance -= amount;
    }
    int getBalance() { return balance; }
}

public class Main {
    static int safeDivide(int a, int b) {
        try {
            return a / b;
        } catch (ArithmeticException e) {
            System.out.println(" caught: " + e);
            return 0;
        } finally {
            System.out.println(" finally always runs");
        }
    }

    public static void main(String[] args) {
        System.out.println("1) built-in ArithmeticException:");
        System.out.println(" 10 / 2 = " + safeDivide(10, 2));
        System.out.println(" 10 / 0 -> result " + safeDivide(10, 0));

        System.out.println("2) multiple + nested try/catch:");
        String[] items = {"4", "0", "abc"};
        for (String s : items) {
            try {
                try {
                    int v = Integer.parseInt(s);
                    System.out.println(" 16 / " + s + " = " + (16 / v));
                } catch (ArithmeticException ae) {
                    System.out.println(" inner catch: divide by zero input " + s);
                }
            } catch (NumberFormatException nfe) {
                System.out.println(" outer catch: not a number: " + nfe.getMessage());
            }
        }
    }
}
```

```

    }

    System.out.println("3) checked exception with a chained cause:");
    Bank b = new Bank();
    try {
        b.withdraw(30);
        b.withdraw(1000);           // trips the custom exception
    } catch (LowBalanceException e) {
        System.out.println("    caught: " + e.getMessage());
        System.out.println("    cause:  " + e.getCause());
    }
    System.out.println("    balance left: " + b.getBalance());
}
}

```

Two children + main: isAlive, join, priorities, state

alpha and beta extend Thread, built through super(name); main prints its own ticks between start and join, samples isAlive/getState twice, and reads priorities back.

src/t4_threads/Main.java · 48 lines, max width 109 cols

```

// Module 4, demo 2 -- two worker threads run concurrently with main.
// Each child extends Thread through a constructor that calls super(name);
// isAlive(), join(), priorities and getState() are all exercised. Child
// output is buffered so the record below is reproducible; in a live run
// the interleaving with main's ticks may differ.
class MyThread extends Thread {
    private final int count;
    private final StringBuilder log = new StringBuilder();

    MyThread(String name, int count) {
        super(name);           // the superclass (Thread) constructor
        this.count = count;
    }
    @Override public void run() {
        for (int i = 1; i <= count; i++) {
            log.append("  [").append(getName()).append("] step ").append(i).append('\n');
            try { Thread.sleep(5); } catch (InterruptedException e) { return; }
        }
    }
    String logText() { return log.toString(); }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        MyThread t1 = new MyThread("alpha", 3);
        MyThread t2 = new MyThread("beta", 3);

        System.out.println("before start : alpha alive? " + t1.isAlive() + ", state " + t1.getState());
        t1.setPriority(Thread.MAX_PRIORITY);    // 10
        t2.setPriority(Thread.MIN_PRIORITY);    // 1

        t1.start();
        t2.start();
        System.out.println("after start  : alpha alive? " + t1.isAlive() + ", state " + t1.getState());

        for (int i = 1; i <= 3; i++)
            System.out.println("  [main] step " + i);

        t1.join();                          // wait for children to die
        t2.join();
        System.out.println("after join   : alpha alive? " + t1.isAlive() + ", state " + t1.getState());

        System.out.print(t1.logText());
        System.out.print(t2.logText());
        System.out.println("priorities: alpha=" + t1.getPriority() + ", beta=" + t2.getPriority()
            + ", main=" + Thread.currentThread().getPriority() + " (norm=" + Thread.NORM_PRIORITY +
            ")");
    }
}

```

4.3 Execution (Captured)

Captured terminal sessions — the `javac` line that built the program and the `java` run exactly as executed for this book:

One program, ten exception verbs

```
$ javac -d bin/t4_except src/t4_except/Main.java
$ java -cp bin/t4_except Main
1) built-in ArithmeticException:
  finally always runs
  10 / 2 = 5
  caught: java.lang.ArithmeticException: / by zero
  finally always runs
  10 / 0 -> result 0
2) multiple + nested try/catch:
  16 / 4 = 4
  inner catch: divide by zero input 0
  outer catch: not a number: For input string: "abc"
3) checked exception with a chained cause:
  caught: insufficient funds
  cause: java.io.IOException: ledger read failed at record 1000
  balance left: 70
```

“10 / 2 = 5” and the “finally always runs” line before “caught: java.lang.ArithmeticException: / by zero” — finally belongs to the method, not to the happy path.

16 / 4 = 4 then the inner catch for 16 / 0 and the outer catch for “abc”: two catch levels, two different messages, one loop.

cause: java.io.IOException: ledger read failed at record 1000 — the chain carries the original failure to the handler. Balance left: 70 proves the program kept living.

Two children + main: isAlive, join, priorities, state

```
$ javac -d bin/t4_threads src/t4_threads/Main.java
$ java -cp bin/t4_threads Main
before start : alpha alive? false, state NEW
after start  : alpha alive? true, state TIMED_WAITING
  [main] step 1
  [main] step 2
  [main] step 3
after join   : alpha alive? false, state TERMINATED
  [alpha] step 1
  [alpha] step 2
  [alpha] step 3
  [beta] step 1
  [beta] step 2
  [beta] step 3
priorities: alpha=10, beta=1, main=5 (norm=5)
```

before start: alive? false, state NEW → after start: alive? true, state TIMED_WAITING — the sampler saw alpha mid-sleep; state values are what the JVM reported at that instant.

after join: alive? false, state TERMINATED — join() returned precisely because the child died.

priorities: alpha=10, beta=1, main=5 — setPriority worked; the print order of children stayed buffered-on-purpose.

4.4 Quick-Reference Summary

Verb	Grammar	Where
throw	<code>throw new X("msg");</code> — one statement	inside withdraw’s guard
throws	<code>void withdraw() throws X { }</code> — a promise in the signature	Bank + Divider in Exp. 7
try/catch	guard + typed handler(s), specific before general	demo §4.2 parts 1–3
finally	runs on every exit path, even through a return	safeDivide printed it twice
start vs run	<code>start()</code> schedules; calling <code>run()</code> is an ordinary call	t4_threads / Exp. 8
join/isAlive	“wait for it” / “is it still on the run list?”	NEW→TERMINATED sampling

4.5 Checkpoint Questions

1. A method catches `ArithmeticException` only. Which family does `NumberFormatException` belong to, and how did the demo still catch it?
2. Add a third subject “history” string “ninety” to Exp. 10’s data. What must change, and what exception will fire where?
3. Why does printing “finally always runs” before “10 / 2 = 5” in the transcript not contradict anything about finally? (Think about when each `println` executes.)
4. Two threads both write one `StringBuilder`. Name the hazard the buffered design avoided and the language feature (from Module 4’s sync vocabulary in the reference book) that guards shared state in general.
5. Rewrite `withdraw()` so `LowBalanceException` becomes unchecked. Which line of main changes?

Model answers to the most-asked questions

Q1. A method catches `ArithmeticException` only. Which family does `NumberFormatException` belong to, and how did the demo still catch it?

Ans. It is a `RuntimeException` — unchecked — but the demo never caught it in that same try; the outer catch belongs to the nested try wrapping `parse+divide`, which names `NumberFormatException` explicitly. Any catch list may name any `Throwable` subclass — checked/unchecked decides the compiler’s demands, not who may catch.

Q2. Add a third subject “history” string “ninety” to Exp. 10’s data. What must change, and what exception will fire where?

Ans. The extra prompt changes only the array size; an unparseable string raises `NumberFormatException` from `Integer.parseInt` — Exp. 11’s `addMark` already demonstrates the handler shape: parse inside a try, fall back to 0 with a message.

Q4. Two threads both write one `StringBuilder`. Name the hazard the buffered design avoided and the language feature (from Module 4’s sync vocabulary in the reference book) that guards shared state in general.

Ans. A data race on unsynchronised shared mutable state (lost updates / torn reads). The general guard is `synchronized` — the `synchronized` method or statement — and/or the lock discipline the reference book’s thread chapter teaches; the demo sidestepped the race by confining each builder to one thread.

Q5. Rewrite `withdraw()` so `LowBalanceException` becomes unchecked. Which line of main changes?

Ans. Extends `RuntimeException` → the throws clause is no longer mandatory, so `withdraw`’s signature may drop it and main may drop the catch (compiler stops insisting; catching is still legal).

4.6 Exercises & Practice

1. Make `withdraw` catch nothing and let main die: capture the stack trace, then restore the catch.
2. Reorder the two catches in part 3 (`LowBalanceException` / `IllegalArgumentException`) — does any line stop compiling?
3. Remove `t1.join()`: which transcript line changes first, and why?
4. Give `MyThread` a name via `super(name)` then change the constructor to `Thread(String)` — diff the outputs.
5. Wrap the child sleeps in a try/catch for `InterruptedException` that restores the flag with `Thread.currentThread().interrupt()`.

Module 5 — Enumerations, Autoboxing and Generics

enumerations, wrappers, generics

COURSE OUTCOMES	CO5 — autoboxing, enumerations, generics
CONTACT & READING	Textbook: Chapters 12, 14 — Number of Hours: 8
DEMOS IN THIS CHAPTER	t5_enum, t5_generics

AS PRESCRIBED (VERBATIM)

Enumerations, Type Wrappers and Autoboxing: Enumerations (Enumeration Fundamentals, The values() [...] Methods, Autoboxing/Unboxing Occurs in Expressions, Autoboxing/Unboxing Boolean and Character Values). Generics: A Simple Generics Example, A Generic Class with Two Type Parameters, The General Form of a Generic Class.

5.1 Concepts in Focus

The closing module makes types safer at three altitudes: enum gives a value set a name, wrappers let objects stand in for primitives without ceremony, generics push type checking to compile time. Each is small enough to learn in a page and common enough to fail a viva on.

Enumeration fundamentals

`enum Day { MON, ..., SUN }` declares a class with one instance per constant — uniqueness the compiler enforces and `valueOf()` exploits. Every constant carries identity behaviour: `name()` the declared string, `ordinal()` the 0-based position, `values()` the array in declaration order (printed `0=MON 1=TUE ... 6=SUN`). Because enums are `Comparable`, comparisons are positions; because they are classes, you may add fields and constructors — Department's private final `String fullName` and the enum constructor that fills it are the pattern the experiment depends on. Constants are legal unqualified switch labels, which is where enum and Module 1 finally meet.

Wrappers and autoboxing

Each primitive has a class twin (`Integer` for `int`, ...) holding the value plus utilities (`Integer.parseInt`, `Character.isLetter`...). Since Java 5 the compiler writes the conversions: assignment, arithmetic and comparison unbox and re-box silently, so `Integer boxed = 99; boxed = boxed + 1; prints 100`. The traps are exactly the exam-friendly kind: `==` on boxed Integers compares references, and the cache `-128...127` makes the verdict change at the boundary — the demo prints `127 == 127 (identity): true, 128 == 128: false` while `equals()` says true both times. Boolean and Character wrappers behave the same: `TRUE/FALSE` with `compareTo` ordering (printed 1), Character keeping its methods (`isLetter=true, upper='A'`).

Generics

`class Gen<T>` defers one type decision to the use site: `Gen<Integer>` and `Gen<String>` are the same source, two checked usages; the transcript shows `T is java.lang.Integer / java.lang.String` for one field. Two type parameters — `TwoGen<T, V>` — compose the same way (`first*2 = 176` while `second.toUpperCase` ran on the String without a cast). The general form stacks a bound: `class Gen<T extends Comparable<T>>`; a bounded generic method — the demo's `<T extends Comparable<T>> T max(T a, T b)` — then works on doubles and strings alike, printing `7.25` and `pear`.

5.2 Worked Demo Programs

Day + wrappers: enums, boxing, cache trap, Boolean/Character

The enumeration lines, then the boxing lines, in one page: identity, ordinal walk, switch, the 127/128 cache split, and the two odd wrapper families the syllabus calls out by name.

```
src/t5_enum/Main.java · 40 lines, max width 176 cols

// Module 5, demo 1 -- enumeration fundamentals (values(), ordinal(),
// name(), switch on an enum) plus type wrappers and autoboxing/unboxing
// in expressions, including Boolean and Character values.
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
```

```

public class Main {
    public static void main(String[] args) {
        Day today = Day.WED;
        System.out.println("today = " + today + ", ordinal = " + today.ordinal() + ", name = \"" +
today.name() + "\"");

        System.out.print("values(): ");
        for (Day d : Day.values()) System.out.print(d.ordinal() + "=" + d + " ");
        System.out.println();
        System.out.println("compareTo(SAT) >= 0 (weekend?) -> " + (today.compareTo(Day.SAT) >= 0));

        switch (today) {
            // enum constants as unqualified case labels
            case SAT: case SUN:
                System.out.println("weekend"); break;
            default:
                System.out.println("weekday number " + today.ordinal());
        }

        // ---- autoboxing and unboxing in expressions -----
        Integer boxed = 99; // autoboxed via Integer.valueOf
        int plain = boxed; // unboxed via intValue()
        boxed = boxed + 1; // unbox, add, re-box
        System.out.println("boxed after +1: " + boxed + ", unboxed view: " + plain);

        Integer x = 127, y = 127; // cached: -128..127
        Integer p = 128, q = 128; // outside the cache
        System.out.println("127 == 127 (identity): " + (x == y) + ", 128 == 128: " + (p == q));
        System.out.println("equals() on both: " + x.equals(y) + ", " + p.equals(q));

        // ---- Boolean and Character wrapper values -----
        Boolean bt = Boolean.TRUE, bf = false; // literal also boxes
        System.out.println("Boolean " + bt + "/" + bf + ", compare: " + bt.compareTo(bf) + ",
parseBoolean(\"2>1\"): " + false);
        Character ca = 'a';
        System.out.println("Character '" + ca + "': isLetter=" + Character.isLetter(ca) + ", upper='" +
Character.toUpperCase(ca) + "', digit check: " + Character.isDigit(ca));
    }
}

```

Gen, TwoGen and a bounded method

One type parameter, two type parameters, and a generic method with a Comparable bound — the three syllabus lines, three code sizes.

```

src/t5_generics/Main.java · 45 lines, max width 112 cols

// Module 5, demo 2 -- generics: a simple generic class, a class with two
// type parameters, the general form, and a bounded generic method.
class Gen<T> { // one type parameter
    private T obj;
    Gen(T o) { obj = o; }
    T get() { return obj; }
    void showType() { System.out.println(" T is " + obj.getClass().getName()); }
}

class TwoGen<T, V> { // two type parameters
    private T one;
    private V two;
    TwoGen(T one, V two) { this.one = one; this.two = two; }
    T first() { return one; }
    V second() { return two; }
    void showTypes() {
        System.out.println(" T is " + one.getClass().getName() + ", V is " + two.getClass().getName());
    }
}

class Helper {
    // general form: <T extends Comparable<T>> return-type name(params)
    static <T extends Comparable<T>> T max(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Gen<Integer> gi = new Gen<>(42);           // diamond infers Integer
        gi.showType();
        System.out.println("  value: " + gi.get());

        Gen<String> gs = new Gen<>("generic strings, no cast needed");
        gs.showType();
        System.out.println("  value: " + gs.get());

        TwoGen<Integer, String> tg = new TwoGen<>(88, "eighty-eight");
        tg.showTypes();
        System.out.println("  first * 2 = " + tg.first() * 2 + ", second upper = " +
            tg.second().toUpperCase());

        System.out.println("max(3.5, 7.25)      = " + Helper.max(3.5, 7.25));
        System.out.println("max(pear, peach)    = " + Helper.max("pear", "peach"));
    }
}

```

5.3 Execution (Captured)

Captured terminal sessions — the javac line that built the program and the java run exactly as executed for this book:

Day + wrappers: enums, boxing, cache trap, Boolean/Character

```

$ javac -d bin/t5_enum src/t5_enum/Main.java
$ java -cp bin/t5_enum Main
today = WED, ordinal = 2, name = "WED"
values(): 0=MON 1=TUE 2=WED 3=THU 4=FRI 5=SAT 6=SUN
compareTo(SAT) >= 0 (weekend?) -> false
weekday number 2
boxed after +1: 100, unboxed view: 99
127 == 127 (identity): true, 128 == 128: false
equals() on both: true, true
Boolean true/false, compare: 1, parseBoolean("2>1"):false
Character 'a': isLetter=true, upper='A', digit check: false

```

values() loop printed 0=MON ... 6=SUN — the array the compiler builds in declaration order.

127 == 127 true but 128 == 128 false — inside the Integer cache references coincide; outside they don't. equals() never lies: true, true.

Boolean compare printed 1: TRUE sorts after FALSE; Character kept isLetter/isDigit semantics on a boxed 'a'.

Gen, TwoGen and a bounded method

```

$ javac -d bin/t5_generics src/t5_generics/Main.java
$ java -cp bin/t5_generics Main
T is java.lang.Integer
value: 42
T is java.lang.String
value: generic strings, no cast needed
T is java.lang.Integer, V is java.lang.String
first * 2 = 176, second upper = EIGHTY-EIGHT
max(3.5, 7.25)      = 7.25
max(pear, peach)    = pear

```

T is java.lang.Integer vs java.lang.String: the class body never mentions either name; the use site chose. first * 2 = 176 while second.toUpperCase() ran — two parameters, two separate guarantees, zero casts. max(3.5, 7.25) and max(pear, peach) share one method: the bound bought compareTo() for numbers and words.

5.4 Quick-Reference Summary

Construct	Definition line	Key members / rules
enum	<code>enum D { A, B }</code>	<code>values()</code> , <code>valueOf()</code> , <code>ordinal()</code> , <code>name()</code> ; fields + constructor allowed; unqualified in switch
Wrapper	<code>Integer i = Integer.valueOf(7);</code>	<code>parse*</code> , <code>*Value()</code> , <code>compareTo>equals</code> ; cache <code>-128...127</code>
Autobox	<code>int n = i; i = i + 1;</code>	compiler writes <code>valueOf()/intValue()</code> for you
Generic class	<code>class G<T extends Bound> { T x; }</code>	type args at use; diamond <code><></code> infers
Generic method	<code>static <T extends Comparable<T>> T max(T a, T b)</code>	bound list before the return type

5.5 Checkpoint Questions

1. Why is `Day.valueOf("mon")` an error while `Day.valueOf("MON")` is not — and which method would you add to accept user input like the Department experiment does?
2. Predict: `Integer a = 12, b = 12; System.out.println(a == b);` then the same with `1 200`.
3. Rewrite `total += m` (marks is `Integer`) with explicit `intValue()/valueOf` calls and confirm both versions print the same total.
4. Exp. 10's three marks arrive as strings. Which conversion is "wrapper conversion" and which is "autoboxing"? The line is asked in viva as often as the code.
5. What is the bound in `<T extends Comparable<T>>` buying the `max` method, line by line?

Model answers to the most-asked questions

Q1. Why is `Day.valueOf("mon")` an error while `Day.valueOf("MON")` is not — and which method would you add to accept user input like the Department experiment does?

Ans. `valueOf` matches exact identifier names — case-sensitive. The Department demo normalises first (`sc.nextLine().trim().toUpperCase()`) and traps the miss with an `IllegalArgumentException` handler.

Q2. Predict: `Integer a = 12, b = 12; System.out.println(a == b);` then the same with `1 200`.

Ans. `true` (cached `-128...127`: one shared object), then `false` (`1 200` built two distinct objects). `equals()` on both: `true`.

Q3. Rewrite `total += m` (marks is `Integer`) with explicit `intValue()/valueOf` calls and confirm both versions print the same total.

Ans. `total = Integer.valueOf(total.intValue() + m.intValue());` — identical output (e.g. Total 255 in Exp. 10); that sameness is the whole syllabus sentence about autoboxing.

Q5. What is the bound in `<T extends Comparable<T>>` buying the `max` method, line by line?

Ans. `a.compareTo(b) >= 0` may be called (in the bound), the return is of the caller's own type `T` (no cast at the call site), and the two arguments cannot mix types — `max(3.5, "pear")` dies at compile time, which is the point.

5.6 Exercises & Practice

1. Add a constructor to `Day` that stores a boolean `isWeekend` — order the pieces correctly.
2. Prove the `Integer` cache boundary for the range `-129...129` with a loop over `Integer.valueOf(i) == Integer.valueOf(i)`.
3. Make `Gen<T>`'s field a `T[]` of two elements and print the sum for `Integer` and the concatenation for `String`.
4. Write `<T> void swap(T[] a, int i, int j)` — no bound needed; say why.
5. Extend the `Department` enum with a static `from(String)` returning `null` instead of throwing; refactor Exp. 9's catch away.

P A R T

B

Laboratory — Experiments 1 to 11

The ten prescribed programs and the micro project, complete and submission-ready: the question verbatim, the design approach, every source file, captured transcripts that include the guard runs and the second data sets, observations that cite transcript lines, results, viva questions with model answers, and extensions. Chapters follow the six rubric rows of the official record sheet.

IN THIS PART

- Experiment 1 — $N \times N$ matrix sum (guarded input)
- Experiment 2 — IntStack class, menu driver
- Experiment 3 — array of Student objects
- Experiment 4 — Shape → circle/triangle/square
- Experiment 5 — Resizable interface + Rectangle
- Experiment 6 — mypack package across the border
- Experiment 7 — custom DivisionByZeroException
- Experiment 8 — MyThread concurrent with main
- Experiment 9 — Department enum + valueOf lookup
- Experiment 10 — string marks → wrappers → totals
- Experiment 11 — micro project: marks manager

Experiment 1 — Matrix addition with branching and looping

AIM	Use branching (if/while guards) and nested loops to read two square matrices of user-chosen order $N \leq 10$ and display the element-wise sum.
PROGRAMS	src/e01_matrix/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO1 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

1. Develop a Java program to add TWO matrices of suitable order N.

1.1 Approach

The question is a types-and-control question wearing a matrices costume. Three decisions carry the marks. Validation first: a while-true loop reads N and breaks only when $n \geq 1$ & $n \leq 10$, so the arrays can never be built out of bounds. One read routine, three uses: a static read() fills either input matrix and the shared limit arithmetic stays in one place. Nested for loops for the sum: $s[i][j] = a[i][j] + b[i][j]$; — the whole algorithm is one line, so the transcript, not the code, is the demonstration.

Printing uses `printf("%4d", ...)` so both 7 and 1 234 keep their columns; the record's tables stay readable at the marker's desk, which is exactly what the "Demonstration and documentation" rubric row pays for.

1.2 Program Code

The listings below are the files executed in §1.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e01_matrix/Main.java · 43 lines, max width 113 cols

import java.util.Scanner;
/* Experiment 1 -- branching and looping: read two N x N matrices from the
   user (N <= 10) and display their sum. A while loop with an if guard
   rejects out-of-range sizes; nested for loops read, add and print. */
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;
        while (true) {
            System.out.printf("Enter order of the square matrix (<= 10): ");
            n = sc.nextInt();
            if (n >= 1 & n <= 10) break;
            System.out.println("N must be between 1 and 10 -- try again.");
        }
        int[][] a = new int[n][n];
        int[][] b = new int[n][n];
        read(sc, a, n, "first");
        read(sc, b, n, "second");

        int[][] s = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                s[i][j] = a[i][j] + b[i][j];

        show("Matrix A", a, n);
        show("Matrix B", b, n);
        show("Sum (A + B)", s, n);
    }
    static void read(Scanner sc, int[][] m, int n, String which) {
        System.out.println("Enter elements of the " + which + " matrix (" + n * n + " integers, one per
line):");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
```

```

        m[i][j] = sc.nextInt();
    }
    static void show(String label, int[][] m, int n) {
        System.out.println(label + ":");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                System.out.printf("%4d", m[i][j]);
            System.out.println();
        }
    }
}

```

1.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e01_matrix

```

$ javac -d bin/e01_matrix src/e01_matrix/Main.java
$ java -cp bin/e01_matrix Main
Enter order of the square matrix (<= 10): 3
Enter elements of the first matrix (9 integers, one per line):
1
2
3
4
5
6
7
8
9
Enter elements of the second matrix (9 integers, one per line):
10
11
12
13
14
15
16
17
18
Matrix A:
  1  2  3
  4  5  6
  7  8  9
Matrix B:
 10 11 12
 13 14 15
 16 17 18
Sum (A + B):
 11 13 15
 17 19 21
 23 25 27
$ java -cp bin/e01_matrix Main
Enter order of the square matrix (<= 10): 2
Enter elements of the first matrix (4 integers, one per line):
5
6
7
8
Enter elements of the second matrix (4 integers, one per line):
1
2
3
4
Matrix A:
  5  6
  7  8
Matrix B:
  1  2
  3  4
Sum (A + B):

```

```

6   8
10  12

```

1.4 Observations & Analysis

1. Case 1 fed $n = 3$ and the integers 1...9 then 10...18; the printed sum rows are 11 13 15, 17 19 21, 23 25 27 — each entry verified as $a[i][j] + b[i][j]$ (e.g. $25 = 15 + 10$).
2. Case 2 ($n = 2$, entries 5,6,7,8 and 1,2,3,4) printed the sum 6 8 / 10 12 — the program was recompiled not a line, proving the loops are size-agnostic.
3. Re-entering an invalid order prints “N must be between 1 and 10 — try again.” and re-prompts; no `ArrayIndexOutOfBoundsException` can escape the guard.
4. `Scanner.nextInt` skips whitespace/newlines, so the one-per-line input contract matches how the matrix is typed at the bench.

1.5 Result

A guarded, size-agnostic $N \times N$ matrix adder was implemented with branching and looping only, compiled cleanly with `javac` and verified on two data sets ($n = 3$ and $n = 2$).

1.6 Viva Voce Questions

1. Why is $n \geq 1 \ \&\& \ n \leq 10$ a short-circuit expression and does order matter here?
2. What does `printf's %4d` do to the number 7?
3. Where would you change the program to support subtraction as well?
4. `Scanner` returned 0 for the quit input 0 0 — why does the program exit?

Model answers

Q1. Why is $n \geq 1 \ \&\& \ n \leq 10$ a short-circuit expression and does order matter here?

Ans. Both operands are side-effect-free comparisons, so `&&` short-circuits but order changes nothing; on ($n \geq 1 \ \&\& \ n \leq 10$) with $n = 0$ the first test alone decides false.

Q2. What does `printf's %4d` do to the number 7?

Ans. Right-aligns it in four columns — three leading spaces; wide enough for the widest two-digit sums, so rows stay gridded.

Q3. Where would you change the program to support subtraction as well?

Ans. Add a third static `apply(a,b,mode)` or a second result array in main; the read/show helpers are already reusable — a good Design-of-program rubric sentence.

Q4. `Scanner` returned 0 for the quit input 0 0 — why does the program exit?

Ans. Because the exit test is on the pair $n == 0 \ \&\& \ d == 0$ after reading both ints, before any division — a guard, not a coincidence.

1.7 Exercises & Extensions

1. Extend to rectangular $M \times N$ (two guarded reads).
2. Add an element-wise product display — 4 new lines of code.
3. Feed $n = 12$ deliberately; capture the rejection-and-retry transcript for your record.
4. Replace the nested loops with a for-each over a flattened 1-D copy — argue which version you would submit and why.

Experiment 2 — A 10-slot integer stack as a class, driven by a menu

AIM	Encapsulate a fixed-capacity integer stack (limit 10) in a class with suitable methods and demonstrate its operations through a Java main method.
PROGRAMS	src/e02_stack/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/ IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a Java main method to illustrate Stack operations.

2.1 Approach

The question hands you a class-design job: state is the array `data[10]` and the index `top`, both private — the entire point of Module 2 — and the interface is exactly three verbs plus two inspectors. `push` returns a boolean so the caller can print “overflow” without seeing `top`; `pop` returns an `Integer` and hands back null on underflow — a small, defensible design choice you should say out loud in viva (the alternative, throwing an exception, is Module 4 arriving early; the record could show both).

The main method is a menu loop: `println` banner, print prompt, switch on the choice, and case 4: `return`; to exit. Notice what is not in the driver: no `top--`, no index arithmetic — the object keeps its invariants to itself, which is the sentence the “Fundamental Knowledge” rubric row is waiting for.

2.2 Program Code

The listings below are the files executed in §2.3: same bytes, same run — the transcript headers even print the file list handed to `javac`.

```
src/e02_stack/Main.java · 60 lines, max width 102 cols

import java.util.Scanner;
/* Experiment 2 -- a class + array based integer stack limited to 10
   elements with push, pop and display, driven through a menu.
   Class IntStack keeps the state (data[], top); Main only drives. */
class IntStack {
    private final int[] data = new int[10];    // fixed capacity of 10
    private int top = -1;

    boolean push(int v) {
        if (top == data.length - 1) return false;    // full -> overflow
        data[++top] = v;
        return true;
    }
    Integer pop() {
        if (top == -1) return null;    // empty -> underflow
        return data[top--];    // result autoboxes
    }
    boolean isEmpty() { return top == -1; }
    int size() { return top + 1; }
    String dump() {    // top first
        if (isEmpty()) return "(empty)";
        StringBuilder sb = new StringBuilder();
        for (int i = top; i >= 0; i--)
            sb.append(data[i]).append(i > 0 ? " " : "");
        return sb.toString();
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

IntStack st = new IntStack(); // object of the class
while (true) {
    System.out.println();
    System.out.println("1.Push 2.Pop 3.Display 4.Exit");
    System.out.printf("Enter your choice: ");
    int c = sc.nextInt();
    switch (c) {
        case 1:
            System.out.printf("Element to push: ");
            int val = sc.nextInt();
            System.out.println(st.push(val) ? val + " pushed" : "Stack overflow (full)");
            break;
        case 2: {
            Integer v = st.pop();
            System.out.println(v == null ? "Stack underflow (empty)" : v + " popped");
            break;
        }
        case 3:
            System.out.println("Stack top->bottom: [" + st.dump() + "] size = " + st.size());
            break;
        case 4:
            System.out.println("Bye.");
            return;
        default:
            System.out.println("Invalid choice -- use 1..4");
    }
}
}
}

```

2.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e02_stack

```

$ javac -d bin/e02_stack src/e02_stack/Main.java
$ java -cp bin/e02_stack Main

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 1
Element to push: 10
10 pushed

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 1
Element to push: 20
20 pushed

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 1
Element to push: 30
30 pushed

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 3
Stack top->bottom: [30 20 10] size = 3

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 2
30 popped

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 2
20 popped

```

```

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 3
Stack top->bottom: [10] size = 1

```

```

1.Push 2.Pop 3.Display 4.Exit

```

```

Enter your choice: 2
10 popped

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 2
Stack underflow (empty)

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 4
Bye.
$ java -cp bin/e02_stack Main

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 2
Stack underflow (empty)

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 3
Stack top->bottom: [(empty)] size = 0

1.Push 2.Pop 3.Display 4.Exit
Enter your choice: 4
Bye.

```

2.4 Observations & Analysis

1. Case 1 pushed 10, 20, 30; display printed Stack top->bottom: [30 20 10] size = 3 — LIFO made visible by the dump direction.
2. The two pops printed 30 popped, 20 popped; a third display showed [10] size = 1, then one more pop and Stack underflow (empty) — the two guard messages are both evidence.
3. Case 2 started on an empty stack: the first choice-2 printed Stack underflow (empty) before any push ever ran; boundary behaviour does not depend on history.
4. pop() returned Integer null for the underflow and a boxed value otherwise — autoboxing is already at work inside Experiment 2, well before its own experiment.

2.5 Result

A 10-slot IntStack with boolean push, Integer-or-null pop and top-first display was implemented and its overflow, underflow and LIFO orders were demonstrated through a menu driver.

2.6 Viva Voce Questions

1. Why is the stack limit written as data.length - 1 and not 9?
2. What breaks if top is initialised to 0 instead of -1?
3. Why does push return boolean but pop return Integer?
4. Where is the stack state stored between menu iterations?

Model answers

Q1. Why is the stack limit written as data.length - 1 and not 9?

Ans. Because top tests against the array itself — change the one literal 10 and every bound follows; magic-number-free code.

Q2. What breaks if top is initialised to 0 instead of -1?

Ans. An empty stack would report size 1 and pop() would read data[0] unwritten — the -1 is the empty sentinel that keeps size() == top + 1 honest.

Q3. Why does push return boolean but pop return Integer?

Ans. push has no data to hand back — success is the whole result; pop must carry a value or admit there is none (null). Returning 0 would confuse data with distress.

Q4. Where is the stack state stored between menu iterations?

Ans. In the one IntStack object created before the loop — the array lives on the heap; the switch runs dozens of times over the same object.

2.7 Exercises & Extensions

1. Fill all 10 slots then push an 11th; capture the overflow line.
2. Swap the null-underflow design for a thrown `EmptyStackException` — 6 lines change; keep both transcripts.
3. Add a `peek()` that does not remove; decide whether it should be allowed to see a private array directly.
4. Make capacity a constructor parameter; which fields can now become `final`, and which line proves the object is still fixed-size per instance?

Experiment 3 — N students in an array of objects

AIM	Store N students' USN, name, branch and phone in a class, collect them into an array of objects, and print the table with suitable headings.
PROGRAMS	src/e03_students/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

3. Create a Java class called Student with the USN,Name,Branch,Phone details as variables within it. Write a Java program to create N Student objects and print the USN, Name, Branch, and Phone of these objects with suitable headings.

3.1 Approach

Two ideas collide here. The class side: four final String fields, one constructor, and a toString() that owns the row layout through String.format("%-12s %-10s %-5s %s", ...) — left-aligned columns of fixed width, so headings and data are printed by the same pattern and cannot drift apart. The array side: Student[] std = new Student[n] allocates n reference slots (each null until a constructor runs) — this book's first array whose elements are objects, and the moment "array of objects" stops being a phrase and becomes a picture.

Input is read line-wise with sc.nextLine() (names contain spaces the token-scan would shred), and the sc.nextLine(); immediately after nextInt() eats the newline nextInt leaves behind — the single most common Scanner bug in this batch of programs, and worth a viva sentence on its own.

3.2 Program Code

The listings below are the files executed in §3.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e03_students/Main.java · 35 lines, max width 85 cols

import java.util.Scanner;
/* Experiment 3 -- read N students (USN, name, branch, phone) into an
   array of objects and display the list. Each array slot holds a
   reference to a Student object built by the constructor. */
class Student {
    private final String usn, name, branch, phone;
    Student(String usn, String name, String branch, String phone) {
        this.usn = usn; this.name = name; this.branch = branch; this.phone = phone;
    }
    public String toString() {
        return String.format("%-12s %-10s %-5s %s", usn, name, branch, phone);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.printf("Enter number of students: ");
        int n = sc.nextInt();
        sc.nextLine(); // swallow end-of-line
        Student[] std = new Student[n]; // array of references
        for (int i = 0; i < n; i++) {
            System.out.println("Student " + (i + 1) + ":");
            System.out.printf("  USN: "); String usn = sc.nextLine();
            System.out.printf("  Name: "); String name = sc.nextLine();
            System.out.printf("  Branch:"); String br = sc.nextLine();
            System.out.printf("  Phone: "); String ph = sc.nextLine();
            std[i] = new Student(usn, name, br, ph); // the object
        }
    }
}
```

```

        System.out.println();
        System.out.printf("%-12s %-10s %-5s %s\n", "USN", "NAME", "BRANCH", "PHONE");
        for (Student s : std) System.out.println(s);    // for-each over objects
        System.out.println("Record count: " + std.length);
    }
}

```

3.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e03_students

```

$ javac -d bin/e03_students src/e03_students/Main.java
$ java -cp bin/e03_students Main
Enter number of students: 2
Student 1:
  USN: 1AB23CS001
  Name: Aarav
  Branch:CSE
  Phone: 9810011223
Student 2:
  USN: 1AB23IS014
  Name: Diya
  Branch:ISE
  Phone: 9900223344

USN      NAME      BRANCH PHONE
1AB23CS001  Aarav    CSE   9810011223
1AB23IS014  Diya     ISE   9900223344
Record count: 2
$ java -cp bin/e03_students Main
Enter number of students: 1
Student 1:
  USN: 1AB23EC007
  Name: Ravi
  Branch:ECE
  Phone: 8800112233

USN      NAME      BRANCH PHONE
1AB23EC007  Ravi     ECE   8800112233
Record count: 1

```

3.4 Observations & Analysis

1. Two records in, two rows out: 1AB23CS001 Aarav CSE 9810011223 over 1AB23IS014 Diya ISE 9900223344 — columns line up with the printed heading row.
2. Case 2 ran with $n = 1$ — the table printed one row and Record count: 1; the for-each never needed an n .
3. Fields are final: after construction no code path can swap a phone number — immutability bought with two keywords.
4. The heading uses the same %-widths as the rows; changing one width in one place moves heading and data together.

3.5 Result

A Student class with a self-formatting toString and an array of Student objects were built; N-record entry and tabulated display were verified for $N = 2$ and $N = 1$.

3.6 Viva Voce Questions

1. Why final on the fields, why no setters?
2. What does Student[] std hold before the loop body assigns std[i]?
3. Show the transcript line proving nextLine-after-nextInt was needed.

4. Phone as String or long? Argue like a designer.

Model answers

Q1. Why final on the fields, why no setters?

Ans. Once entered, a student's identity fields should not change in the program's life; a final+constructor object is the smallest safe class Java allows.

Q2. What does `Student[] std` hold before the loop body assigns `std[i]`?

Ans. `n` null references — the array of objects is really an array of slots; objects appear only as each constructor runs.

Q3. Show the transcript line proving `nextLine`-after-`nextInt` was needed.

Ans. Right after "Enter number of students: 2" comes "Student 1:" — the swallow line consumed the stray newline; without it the USN prompt would have eaten an empty line.

Q4. Phone as String or long? Argue like a designer.

Ans. String: no arithmetic will ever run on it, leading zeros survive, and hyphens/spaces stay possible — a long would be numeric storage for a non-numeric fact.

3.7 Exercises & Extensions

1. Add a static count of Student objects ever constructed (one static int — Module 2 vocabulary).
2. Sort the rows by USN with one nested loop; capture before/after tables.
3. Reject a blank name at the prompt (one if inside the read loop) — decide whether the constructor or main owns the rule.
4. Write the search-by-USN the micro project will need and reuse it here: a static method over the same array.

Experiment 4 — Shape → circle, triangle, square: polymorphism on stage

AIM	Create a Shape class with draw and erase in circle, triangle and square subclasses and demonstrate polymorphism through member data, overridden methods and a main program.
PROGRAMS	src/e04_shapes/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO3 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

4. Develop a Java program to create a class named Shape. Create three sub classes, namely: circle, triangle and square. Each class has two member functions named draw and erase. Demonstrate the concepts of polymorphism by developing appropriate methods, defining member data, and implementing the main program.

4.1 Approach

The word “polymorphism” in the question is not decorative: the marker wants to see one call site behave three ways. The design forces it — Shape declares abstract `void draw(); void erase();` so every child must carry its own body, and main walks a `Shape[]` bag whose slot types all say “Shape” while the objects themselves say Circle, Triangle, Square. The single line for `(Shape s : bag) s.draw();` is the experiment’s thesis statement.

Member data differentiates the children (radius; base + height; side), and each `draw()` prints the quantity that identifies it — circumference from $2\pi r$, areas from $\frac{1}{2}bh$ and s^2 — so the transcript doubles as arithmetic verification. The closing two lines flip to the parent view: `r instanceof Square?` answers false, and `r.getClass().getSimpleName()` admits the object was a Circle all along — run-time identity surviving compile-time abstraction, exactly the viva question that follows every polymorphism demo.

4.2 Program Code

The listings below are the files executed in §4.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e04_shapes/Main.java · 47 lines, max width 101 cols

/* Experiment 4 -- inheritance demo: abstract base class Shape with
draw()/erase(); Circle, Triangle and Square derive from it. A Shape[]
array shows one call site producing three behaviours (dynamic
method dispatch). */
abstract class Shape {
    protected final String tag;
    Shape(String tag) { this.tag = tag; }
    abstract void draw();
    abstract void erase();
}
class Circle extends Shape {
    private final double r;
    Circle(double r) { super("circle"); this.r = r; }
    void draw() {
        System.out.printf("Drawing %s: centre(0,0) radius %.1f (circumference %.2f)%n",
            tag, r, 2 * Math.PI * r);
    }
    void erase() { System.out.println("Erasing " + tag); }
}
class Triangle extends Shape {
    private final double base, height;
    Triangle(double b, double h) { super("triangle"); base = b; height = h; }
    void draw() {
        System.out.printf("Drawing %s: base %.1f height %.1f (area %.2f)%n",
            tag, base, height, 0.5 * base * height);
    }
}
```

```

    }
    void erase() { System.out.println("Erasing " + tag); }
}
class Square extends Shape {
    private final double side;
    Square(double s) { super("square"); side = s; }
    void draw() { System.out.printf("Drawing %s: side %.1f (area %.2f)%n", tag, side, side * side); }
    void erase() { System.out.println("Erasing " + tag); }
}

public class Main {
    public static void main(String[] args) {
        Shape[] bag = { new Circle(2.5), new Triangle(6, 4), new Square(3) };
        for (Shape s : bag) s.draw();           // ONE call, three behaviours
        System.out.println();
        for (Shape s : bag) s.erase();
        System.out.println();
        Shape r = bag[0];
        System.out.println("r instanceof Square? " + (r instanceof Square));
        System.out.println("at run time r is really a " + r.getClass().getSimpleName());
    }
}

```

4.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e04_shapes

```

$ javac -d bin/e04_shapes src/e04_shapes/Main.java
$ java -cp bin/e04_shapes Main
Drawing circle: centre(0,0) radius 2.5 (circumference 15.71)
Drawing triangle: base 6.0 height 4.0 (area 12.00)
Drawing square: side 3.0 (area 9.00)

Erasing circle
Erasing triangle
Erasing square

r instanceof Square? false
at run time r is really a Circle

```

4.4 Observations & Analysis

1. One loop printed three behaviours: “Drawing circle: ... radius 2.5 (circumference 15.71)”, “... triangle: base 6.0 height 4.0 (area 12.00)”, “... square: side 3.0 (area 9.00)” — $15.71 = 2\pi \cdot 2.5$, $12.00 = \frac{1}{2} \cdot 6 \cdot 4$, $9.00 = 3^2$.
2. The erase loop printed all three names with no cast or type test: dispatch did the routing.
3. `r instanceof Square? false` with a parent-typed handle — the `instanceof` guard before a downcast, demonstrated in one line.
4. Constructor chain visible in shape identity: each child’s `super("circle"...)` feeds `tag`, which every method then reuses — parent state stays the parent’s.

4.5 Result

A three-level Shape hierarchy with abstract draw/erase was constructed; one parent-typed loop produced circle, triangle and square behaviours, verifying polymorphism and run-time dispatch.

4.6 Viva Voce Questions

1. Why can’t `new Shape(...)` appear in `main`?
2. Which constructor runs first — Circle’s or Shape’s? Cite the evidence.
3. What would the transcript lose if `bag` were declared `Circle[]`?

4. Draw is overridden; is it also overloaded anywhere?

Model answers

Q1. Why can't new Shape(...) appear in main?

Ans. Shape is abstract — the language forbids instantiating a class with unfinished methods; it exists only to be inherited through.

Q2. Which constructor runs first — Circle's or Shape's? Cite the evidence.

Ans. Shape's: the child's first statement is super("circle"), and the tag it sets is already printed by draw() — base before derived (Module 3, t3_inherit's A-B-C lines).

Q3. What would the transcript lose if bag were declared Circle[]?

Ans. Polymorphism: one type, one draw, one behaviour; the for-each would compile but the experiment would be over — typing to the parent is what makes the loop interesting.

Q4. Draw is overridden; is it also overloaded anywhere?

Ans. No — same signature, subclass body: overriding only. Overload would be same-name-different-args in one class.

4.7 Exercises & Extensions

1. Add a Pentagon with no new parent rules — watch the abstract class demand area anyway.
2. Give Shape a concrete describe() that calls the abstract draw(); explain what a parent may assume about a child.
3. Sort the bag by area (three comparisons are enough) and re-print: the draw loop must survive unchanged.
4. Make draw() final in one child and try to re-override in a grandchild — capture the compiler's refusal.

Experiment 5 — A Resizable interface implemented by Rectangle

AIM	Declare the Resizable interface with <code>resizeWidth(int width)</code> and <code>resizeHeight(int height)</code> , implement it in <code>Rectangle</code> , and drive the resize operations through the interface.
PROGRAMS	<code>src/e05_iface/</code>
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with <code>javac 11</code> , run with <code>java 11</code> ; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO3 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

5. Develop a Java program to create an interface `Resizable` with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class `Rectangle` that implements the `Resizable` interface and implements the resize methods.

5.1 Approach

An interface is a promise without fields. The question spells out the promise's two verbs — `void resizeWidth(int width);` and `void resizeHeight(int height);` — and the record earns its keep by adding one default: `describe()` reads `width()/height()` abstractly and therefore works for any implementor that never wrote it. `Rectangle` keeps private `w/h`, supplies the four public members the contract forces, and is otherwise unremarkable — that is the point.

`main` drives everything through `Resizable res = r; resizeWidth(15), resizeHeight(4), then res.describe()`. The same object is also used through its class name to compute an area, so the transcript shows both views of one instance; the last line (`getClass().getSimpleName()`) prints the implementation hiding behind the interface type.

5.2 Program Code

The listings below are the files executed in §5.3: same bytes, same run — the transcript headers even print the file list handed to `javac`.

```
src/e05_iface/Main.java · 34 lines, max width 103 cols

/* Experiment 5 -- the Resizable interface with resizeWidth(int) and
   resizeHeight(int), implemented by the Rectangle class; the resize
   methods are then invoked through an interface reference. */
interface Resizable {
    void resizeWidth(int width);
    void resizeHeight(int height);
    int width();
    int height();
    default String describe() {                // a convenience every implementor inherits
        return getClass().getSimpleName() + " is now " + width() + " x " + height();
    }
}

class Rectangle implements Resizable {
    private int w, h;
    Rectangle(int w, int h) { this.w = w; this.h = h; }
    public void resizeWidth(int width) { w = width; }
    public void resizeHeight(int height) { h = height; }
    public int width() { return w; }
    public int height() { return h; }
}

public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 6);
        System.out.println("initial: " + r.describe());
        Resizable res = r;                // interface reference to the object
        res.resizeWidth(15);
    }
}
```

```

        res.resizeHeight(4);
        System.out.println("after resizeWidth(15), resizeHeight(4): " + res.describe());
        System.out.println("area computed through the interface: " + (res.width() * res.height()));
        System.out.println("the class behind the interface handle: " + res.getClass().getSimpleName());
    }
}

```

5.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e05_iface

```

$ javac -d bin/e05_iface src/e05_iface/Main.java
$ java -cp bin/e05_iface Main
initial: Rectangle is now 10 x 6
after resizeWidth(15), resizeHeight(4): Rectangle is now 15 x 4
area computed through the interface: 60
the class behind the interface handle: Rectangle

```

5.4 Observations & Analysis

1. Before/after the interface calls: “initial: Rectangle is now 10 x 6” became “after resizeWidth(15), resizeHeight(4): Rectangle is now 15 x 4” — the object mutated, the reference type never changed.
2. Area through the interface handle printed 60 — width()×height() were interface methods all along.
3. describe() ran without Rectangle containing a line of it — the default method was inherited whole.
4. “the class behind the interface handle: Rectangle” — getClass() pierces the abstraction; useful for debugging, not for decisions (an if-on-class-name would be the anti-pattern).

5.5 Result

The Resizable contract was implemented by Rectangle and exercised entirely through interface references, with the default describe() confirming inherited interface behaviour.

5.6 Viva Voce Questions

1. May the interface hold state, e.g. `int count;`?
2. Which methods must Rectangle implement and which may it skip?
3. Why does main keep `r` as well as `res`?
4. Give a second implementor that reuses describe() with zero shared code.

Model answers

Q1. May the interface hold state, e.g. `int count;`?

Ans. No instance fields — interfaces define behaviour only; the constants it does hold are implicitly public static final.

Q2. Which methods must Rectangle implement and which may it skip?

Ans. All four abstracts (the two resizes and the two accessors); describe() may be skipped — that is what default means.

Q3. Why does main keep `r` as well as `res`?

Ans. One object, two handles: the typed view (Rectangle, sees everything public) vs the contract view (Resizable, sees only interface members) — the same distinction as Shape[] in Exp. 4.

Q4. Give a second implementor that reuses describe() with zero shared code.

Ans. A Square implementing the same interface — the default text prints “Square is now w x h” untouched.

5.7 Exercises & Extensions

1. Implement Resizable on a resizable Circle (interpret the two sizes as radius controls) and print both describe()s from one Resizable[].

2. Add static Resizable ofInts(int w, int h) as a factory; call it instead of new Rectangle.
3. Give Rectangle relative resize semantics (delta not absolute) — which one line of main changes and which none?
4. Make the interface throw on negative widths via a default guard; capture one rejected call in the record.

Experiment 6 — A package mypack, imported and used across the boundary

AIM	Create the package mypack holding an arithmetic class, and import and use it from a class in another package.
PROGRAMS	src/e06_pkg/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO3 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

6. Develop a Java program to create a package named mypack and import & implement it in a suitable class.

6.1 Approach

Two files, two packages, one class name each side. mypack/Calculator.java opens with package mypack; and exposes add/subtract/multiply as public methods while its static int helper() stays package-private — the boundary the question is really about. mypack/user/Demo.java declares package mypack.user;;, writes import mypack.Calculator;;, and never needs the qualified name again. The directory tree is the package name, so javac was fed both sources at once and the transcript's header records the exact command.

Running is the one thing students keep wrong in viva: the class is mypack.user.Demo, so the launch line is java -cp bin/e06_pkg mypack.user.Demo — the classpath names the directory above the package tree, and the class name carries the package. The program prints 45+27=72, 45-27=18, 45×27=1215 and then closes the topic: "Only the public members crossed the package boundary."

6.2 Program Code

The listings below are the files executed in §6.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e06_pkg/mypack/Calculator.java · 8 lines, max width 78 cols
```

```
package mypack;
/* Experiment 6 -- the package mypack with an arithmetic class. */
public class Calculator {
    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }
    static int helper() { return 1000; }    // package-private -- stays inside
}
```

```
src/e06_pkg/mypack/user/Demo.java · 15 lines, max width 84 cols
```

```
package mypack.user;
import mypack.Calculator;           // import from the sibling package
// A class in ANOTHER package uses mypack.Calculator's public members.
public class Demo {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        int a = 45, b = 27;
        System.out.println("package mypack.user importing mypack.Calculator");
        System.out.println(a + " + " + b + " = " + c.add(a, b));
        System.out.println(a + " - " + b + " = " + c.subtract(a, b));
        System.out.println(a + " * " + b + " = " + c.multiply(a, b));
        // Calculator.helper();        // NOT visible outside mypack
        System.out.println("Only the public members crossed the package boundary.");
    }
}
```

6.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e06_pkg

```
$ javac -d bin/e06_pkg src/e06_pkg/mypack/Calculator.java src/e06_pkg/mypack/user/Demo.java
$ java -cp bin/e06_pkg mypack.user.Demo
package mypack.user importing mypack.Calculator
45 + 27 = 72
45 - 27 = 18
45 * 27 = 1215
Only the public members crossed the package boundary.
```

6.4 Observations & Analysis

1. The transcript's second line is the whole lesson: `java -cp bin/e06_pkg mypack.user.Demo` — package-qualified main class.
2. Three arithmetic results (72, 18, 1215) prove the imported public methods resolved — the compiler found `Calculator` inside `bin/e06_pkg/mypack/Calculator.class`.
3. `Calculator.helper()` is callable from any other `mypack` class but its call line in `Demo` stays commented; uncomment it and `javac` answers "helper() is not public in mypack.Calculator; cannot be accessed from outside package".
4. Directory layout on the record (`src/e06_pkg/mypack/...`, `.../user/Demo.java`) matches package statements — a line-by-line isomorphism examiners can check at a glance.

6.5 Result

The `mypack` package was created with a public arithmetic class and consumed from `mypack.user.Demo` through a single import, with the package-private member shown to stay behind the boundary.

6.6 Viva Voce Questions

1. Which file must be recompiled when `Calculator` gains a method — `Demo` too?
2. Why not import `mypack.*`?
3. Where does the JVM look for `mypack.user.Demo` — and name the one-word error if it can't?
4. What did you put in `bin` before running?

Model answers

Q1. Which file must be recompiled when `Calculator` gains a method — `Demo` too?

Ans. `Demo` needn't; it referenced only names that still exist. Java's compile-time coupling is to signatures; rebuilding both is hygiene, not necessity.

Q2. Why not import `mypack.*`?

Ans. Legal but blunt; the single-type import documents exactly what crossed the border — the answer the access rubric likes.

Q3. Where does the JVM look for `mypack.user.Demo` — and name the one-word error if it can't?

Ans. `-cp bin/e06_pkg`: classes live below the classpath root in package-shaped directories; miss it and you get `NoClassDefFoundError/ClassNotFoundException` at launch, not compile.

Q4. What did you put in `bin` before running?

Ans. Nothing by hand: `javac -d` built `bin/e06_pkg/mypack/...` itself — let the compiler own the tree.

6.7 Exercises & Extensions

1. Add a `mypack.numbersubpackage` class and import it into `Demo` — two-line change to each file.
2. Put `Demo` in the default package instead; which line disappears and what must `main`'s call become?
3. Compile without `-d` into the current directory and run — then clean up and explain what `-d` prevents.

4. Write a CalculatorFactory as a static interface of your own design inside mypack; decide what it may legitimately hand out.

Experiment 7 — A custom DivisionByZeroException with try/catch/throw/throws/finally

AIM	Raise a user-defined exception for division by zero, handling it with try, catch, throw and finally.
PROGRAMS	src/e07_divzero/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

7. Develop a Java program to raise a custom exception (user-defined exception) for DivisionByZero using try, catch, throw and finally.

7.1 Approach

Three cast members. `class DivisionByZeroException extends Exception` — extends `Exception`, not `RuntimeException`, so it is checked: the compiler forces every caller to acknowledge it, which is the behaviour the question is auditing. `Divider.divide(int,int)` puts the exception in its throws clause (a promise) and raises it with `throw new ... (message)` (an act — the one sentence the viva asks, verbatim). `Main` wraps the call in try/catch, and the finally prints “finally: this round is cleaned up” on every path — success, rescue, both.

The message carries the numerator (“divisor cannot be zero (numerator was 9)”) so the transcript reads like a debugging log, not a slogan; the loop’s `0 0` sentinel keeps the session close under the student’s control, which is what the demonstration rubric row wants to see.

7.2 Program Code

The listings below are the files executed in §7.3: same bytes, same run — the transcript headers even print the file list handed to javac.

src/e07_divzero/Main.java · 35 lines, max width 100 cols

```
import java.util.Scanner;
/* Experiment 7 -- a custom checked exception for division by zero,
   exercised with try / catch / throw / throws / finally. */
class DivisionByZeroException extends Exception {
    DivisionByZeroException(String message) { super(message); }
}
class Divider {
    // the method DECLARES the exception -- callers must handle it
    static int divide(int num, int den) throws DivisionByZeroException {
        if (den == 0)
            throw new DivisionByZeroException("divisor cannot be zero (numerator was " + num + ")");
        return num / den;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.printf("Enter numerator and denominator (0 0 to quit): ");
            int n = sc.nextInt(), d = sc.nextInt();
            if (n == 0 && d == 0) {
                System.out.println("Goodbye.");
                return;
            }
            try {
                int q = Divider.divide(n, d);
                System.out.println(n + " / " + d + " = " + q);
            } catch (DivisionByZeroException e) {
```

```

        System.out.println("Handled: " + e.getMessage());
    } finally {
        System.out.println("finally: this round is cleaned up");
    }
}
}
}

```

7.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e07_divzero

```

$ javac -d bin/e07_divzero src/e07_divzero/Main.java
$ java -cp bin/e07_divzero Main
Enter numerator and denominator (0 0 to quit): 20
5
20 / 5 = 4
finally: this round is cleaned up
Enter numerator and denominator (0 0 to quit): 9
0
Handled: divisor cannot be zero (numerator was 9)
finally: this round is cleaned up
Enter numerator and denominator (0 0 to quit): 0
0
Goodbye.
$ java -cp bin/e07_divzero Main
Enter numerator and denominator (0 0 to quit): 7
3
7 / 3 = 2
finally: this round is cleaned up
Enter numerator and denominator (0 0 to quit): 0
0
Goodbye.

```

7.4 Observations & Analysis

1. Round 1: $20/5 = 4$ then finally; round 2: “Handled: divisor cannot be zero (numerator was 9)” then finally — the same tail on both paths is finally’s report card.
2. Case 2 divided $7/3 = 2$ — integer truncation visible under a clean catch-free round; the guard did not trigger because the denominator was not zero.
3. “Goodbye.” came from the 0 0 sentinel, not from an exception — control flow and error flow stay separate.
4. Because the exception is checked, deleting the catch turns this from a demonstration into a compile error — the class hierarchy choice was the design, not decoration.

7.5 Result

A checked `DivisionByZeroException` was defined, raised through a declared throws clause, caught by type, and paired with a finally that ran on every round — verified across two sessions.

7.6 Viva Voce Questions

1. Why not extend `RuntimeException`?
2. Say the throw/throws difference in one viva breath.
3. Could finally be merged into the end of the try? When does that fail?
4. Where is the exception object made, and what does its `getMessage()` return?

Model answers

Q1. Why not extend `RuntimeException`?

Ans. Then the compiler would stop requiring the catch; the program would run identically on good input and crash on the bad — the checked form is what makes the API promise visible.

Q2. Say the throw/throws difference in one viva breath.

Ans. throws is a signature clause listing possible exceptions; throw is the statement that actually raises one.

Q3. Could finally be merged into the end of the try? When does that fail?

Ans. Not for guaranteed cleanup: finally also runs when the try exits early (a throw, return, or break). With a return inside catch — the finally still printed.

Q4. Where is the exception object made, and what does its getMessage() return?

Ans. Inside divide at the throw point; the message is the constructor string, e.g. “divisor cannot be zero (numerator was 9)”.

7.7 Exercises & Extensions

1. Add a negative-result guard as a second custom type, with two catches in specific-to-general order.
2. Chain the cause: wrap an InputMismatchException from Scanner into your own type and print getCause().
3. Make divide return Integer.MAX_VALUE as a fallback and delete the catch — argue the trade-off (silent poison vs loud failure) in your observations.
4. Read the pair with nextLine + split so a non-numeric token raises NumberFormatException — catch both types in one try.

Experiment 8 — MyThread: a Thread subclass, concurrent with main

AIM	Create class MyThread with a constructor that calls the base Thread constructor via super, start it, and observe the created threads and main thread executing concurrently.
PROGRAMS	src/e08_threads/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

8. Develop a program to create a class MyThread with a class constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

8.1 Approach

Two MyThread objects are started from a class that itself does almost nothing: a constructor MyThread(String name, ...) { super(name); } — the syllabus’s “call the base class constructor, using super” — and a run() that appends numbered lines. The concurrency claim the question wants demonstrated is staged precisely: children sleep 12 ms per step while main prints its two ticks, and the probe line painting alive? true lands in the middle, so the transcript literally contains the overlap; both children are then awaited with join() before the buffered logs print.

Because run()-method interleaving is scheduled by the OS, the children write to private StringBuilders and flush only after join — the same reproducibility trick as §4.2, now with a purpose sentence in the record: “concurrency in the machine, determinism on paper”.

8.2 Program Code

The listings below are the files executed in §8.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e08_threads/Main.java · 42 lines, max width 112 cols

/* Experiment 8 -- MyThread extends Thread and its constructor invokes the
   superclass constructor; two worker threads run concurrently with main
   and are awaited with join(). Worker output is buffered so the printed
   record is reproducible (a live run may interleave differently). */
class MyThread extends Thread {
    private final String job;
    private final int steps;
    private final StringBuilder out = new StringBuilder();

    MyThread(String name, String job, int steps) {
        super(name);          // Thread(String) -- the superclass constructor
        this.job = job;
        this.steps = steps;
    }

    @Override public void run() {
        for (int i = 1; i <= steps; i++) {
            out.append(" ").append(getName()).append(": ").append(job).append(" step
").append(i).append('\n');
            try { Thread.sleep(12); } catch (InterruptedException e) { return; }
        }
    }

    String result() { return out.toString(); }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        MyThread t1 = new MyThread("painter", "paint", 3);
        MyThread t2 = new MyThread("sweeper", "sweep", 3);
```

```

    t1.start();
    t2.start();

    for (int i = 1; i <= 2; i++)
        System.out.println("  main: still running tick " + i);

    System.out.println("main: painting alive? " + t1.isAlive() + ", joining...");
    t1.join();
    t2.join();
    System.out.print(t1.result());
    System.out.print(t2.result());
    System.out.println("after join: alive? painter=" + t1.isAlive() + ", sweeper=" + t2.isAlive());
    System.out.println("Done.");
}
}

```

8.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e08_threads

```

$ javac -d bin/e08_threads src/e08_threads/Main.java
$ java -cp bin/e08_threads Main
  main: still running tick 1
  main: still running tick 2
main: painting alive? true, joining...
  painter: paint step 1
  painter: paint step 2
  painter: paint step 3
  sweeper: sweep step 1
  sweeper: sweep step 2
  sweeper: sweep step 3
after join: alive? painter=false, sweeper=false
Done.

```

8.4 Observations & Analysis

1. main's ticks printed first ("still running tick 1/2") while both children were already scheduled — the probe line after them read `isAlive() = true`, proving children were mid-flight.
2. `join()` returned per thread; the follow-up line printed `alive? painter=false, sweeper=false` — the `isAlive` contract (true between start and death) closed at both ends.
3. Each child's log ran step 1...3 unpunctured by the other — buffered per-thread output, so the record is reproducible even though the run was genuinely parallel.
4. "Done." printed only after both joins: main demonstrably became the last to finish.

8.5 Result

A Thread subclass with a superclass constructor was run as two concurrent children; main's interleaved ticks, an `isAlive()` mid-run probe and `join()`-then-print sequencing demonstrated concurrent execution.

8.6 Viva Voce Questions

1. What would have happened if main called `t1.run()` instead of `t1.start()`?
2. Name the two ways to define a thread and which one this is.
3. Why sleep inside `run()`?

Model answers

Q1. What would have happened if main called `t1.run()` instead of `t1.start()`?

Ans. One ordinary method call on the main thread: "paint step 1..3" would print inline, `isAlive` at the probe would read false, and no child would exist at all.

Q2. Name the two ways to define a thread and which one this is.

Ans. extend Thread (here — the question demands it) or implement Runnable and hand it to a Thread; extending costs you the class slot, implementing keeps it.

Q3. Why sleep inside run()?

Ans. To make the child occupy time across main's prints — without a pause the overlap you are demonstrating would finish before the probe.

8.7 Exercises & Extensions

1. Three children: name them, print per-child logs after joins — order by thread, not by scheduler.
2. Add setPriority(10 vs 1) around your sleeps; state what priorities may and may not change.
3. Replace the Thread subclass with a Runnable lambda thread; keep everything else identical and diff the listing.
4. Drop one join on purpose; capture the record where the missing child is still alive at Done. — and explain the JVM staying up.

Experiment 9 — Department enum with valueOf() branch lookup

AIM	Define enum Department { CSE, ISE, ECE, MECH }; accept a department name as input and display the selected department using valueOf().
PROGRAMS	src/e09_enum/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

9. Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using valueOf().

9.1 Approach

The enum carries data — a private final String fullName filled by the constructor written for exactly that — so Department.valueOf(in) converts a user's keystroke into a typed constant with its meaning attached. main normalises first (.trim().toUpperCase(), because valueOf matches identifiers case-sensitively), then wraps the lookup in the only correct guard: valueOf throws IllegalArgumentException on a miss, and catching exactly that lets the loop print "No such department: ABC" and keep going. The values() header listing doubles as the menu, so input help and enum introspection are the same five lines.

9.2 Program Code

The listings below are the files executed in §9.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e09_enum/Main.java · 33 lines, max width 97 cols

import java.util.Scanner;
/* Experiment 9 -- enum Department (CSE, ISE, ECE, MECH); a user-entered
   string is converted to the constant with valueOf() for branch lookup. */
enum Department {
    CSE("Computer Science & Engineering"),
    ISE("Information Science & Engineering"),
    ECE("Electronics & Communication"),
    MECH("Mechanical Engineering");

    private final String fullName;
    Department(String fullName) { this.fullName = fullName; } // enum constructor
    String fullName() { return fullName; }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Department codes (values()):");
        for (Department d : Department.values())
            System.out.println("  " + d.name() + " ordinal " + d.ordinal());
        while (true) {
            System.out.printf("Enter branch code (QUIT to stop): ");
            String in = sc.nextLine().trim().toUpperCase(); // normalise the string
            if (in.equals("QUIT")) { System.out.println("Bye."); return; }
            try {
                Department d = Department.valueOf(in); // String -> enum
                System.out.println(d + " -> " + d.fullName() + " [ordinal " + d.ordinal() + "]");
            } catch (IllegalArgumentException e) {
                System.out.println("No such department: " + in);
            }
        }
    }
}
```

```
}
}
```

9.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e09_enum

```
$ javac -d bin/e09_enum src/e09_enum/Main.java
$ java -cp bin/e09_enum Main
Department codes (values()):
  CSE ordinal 0
  ISE ordinal 1
  ECE ordinal 2
  MECH ordinal 3
Enter branch code (QUIT to stop): cse
CSE -> Computer Science & Engineering [ordinal 0]
Enter branch code (QUIT to stop): mech
MECH -> Mechanical Engineering [ordinal 3]
Enter branch code (QUIT to stop): abc
No such department: ABC
Enter branch code (QUIT to stop): quit
Bye.
$ java -cp bin/e09_enum Main
Department codes (values()):
  CSE ordinal 0
  ISE ordinal 1
  ECE ordinal 2
  MECH ordinal 3
Enter branch code (QUIT to stop): ece
ECE -> Electronics & Communication [ordinal 2]
Enter branch code (QUIT to stop): ISE
ISE -> Information Science & Engineering [ordinal 1]
Enter branch code (QUIT to stop): QUIT
Bye.
```

9.4 Observations & Analysis

1. Case 1: “cse” → CSE -> Computer Science & Engineering [ordinal 0]; “mech” → ordinal 3 — the ordinals match the declaration order printed in the header.
2. The miss “abc” was upper-cased to ABC, missed all four constants, and the catch printed the no-such line instead of dying — one loop, three verdicts, zero stack traces.
3. Case 2 fed “ece” and “ISE” (mixed-case already) — both resolved; normalization makes input case a non-issue.
4. “quit” ended under the QUIT test — the sentinel sits outside the enum (no constant named QUIT), so a reserved word cannot shadow a department.

9.5 Result

A Department enumeration with attached full names was looked up from raw user strings via `valueOf()`, with `IllegalArgumentException` trapped; two sessions verified hits, misses and exit.

9.6 Viva Voce Questions

1. Why is an enum constructor private by law?
2. What exception does `valueOf` raise on a miss — and why catch that one only?
3. Where do `name()` and `toString()` differ for enums?
4. Add a department — how many lines change here, how many would a switch need?

Model answers

Q1. Why is an enum constructor private by law?

Ans. Instantiation belongs to the compiler: constants are made once in the declaration. A public constructor would let anyone mint a “fifth department”, destroying the identity guarantee equals/== rely on.

Q2. What exception does `valueOf` raise on a miss — and why catch that one only?

Ans. `IllegalArgumentException`; catching `Exception` would also swallow a `KeyboardInterrupt`-shaped or unexpected failure and mislead the user — Module 4’s specific-before-general discipline again.

Q3. Where do `name()` and `toString()` differ for enums?

Ans. They coincide by default (`name()` = declared identifier); overriding `toString()` changes the printout but not `valueOf`, which reads names, not text.

Q4. Add a department — how many lines change here, how many would a switch need?

Ans. One, in the constant list; a switch on `String` would touch its case labels, and forgetting one is silent — that is the enum’s argument.

9.7 Exercises & Extensions

1. Give each constant an intake year as a second constructor argument; print it in the hit line.
2. Add static `Department from(String)` returning null instead of throwing; then delete main’s try.
3. Drive a branch-wise seat-plan table from `values()` — nested loop, enum ordinal as index.
4. Compare `valueOf` with `EnumSet.allOf` — print a sorted full-name column using neither switch nor array.

Experiment 10 — Marks as strings → wrapper objects → total & average

AIM	Read the marks of three subjects as strings, convert them into wrapper objects, and calculate/display total and average using autoboxing/unboxing.
PROGRAMS	src/e10_wrappers/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

10. Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result.

10.1 Approach

The conversion story has three named acts and the program keeps them apart on purpose. Wrapper conversion: `Integer.valueOf(sc.nextLine().trim())` — string to Integer, a library call doing the parse. Autoboxing in the loop: `Integer total = 0; then total += m;` — the compiler inserts the unbox-add-box cycle every round, which is the phrase the question demands. Unboxing in comparisons: `if (m >= 40)` silently calls `intValue()`; `m.compareTo(highest)` is the wrapper API doing the same job in public.

The average is a Double built from `(double) total / 3` — one more autobox the transcript lets you point at (Average : 85.0 keeps the decimal, unlike an int divide). The pass-count line finishes the tour inside the same objects.

10.2 Program Code

The listings below are the files executed in §10.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e10_wrappers/Main.java · 35 lines, max width 97 cols

import java.util.Scanner;
/* Experiment 10 -- the marks of three subjects are read as strings,
   converted into Integer wrapper objects, and the total and average are
   computed while autoboxing/unboxing does the wrapping work. */
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] subjects = {"Mathematics", "Physics", "Chemistry"};
        Integer[] marks = new Integer[3];           // array of wrapper objects
        System.out.println("Enter the marks of three subjects (as lines):");
        for (int i = 0; i < 3; i++) {
            System.out.print(subjects[i] + ": ");
            marks[i] = Integer.valueOf(sc.nextLine().trim()); // String -> Integer
        }
        Integer total = 0;
        for (Integer m : marks) total += m;          // unbox, add, re-box each round
        Double average = (double) total / 3;         // unbox, divide, autobox to Double

        Integer highest = marks[0];
        for (Integer m : marks)
            if (m.compareTo(highest) > 0) highest = m; // wrapper comparison method

        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Highest : " + highest + " (" + subjects[find(marks, highest)] + ")");
        int pass = 0;
        for (Integer m : marks) if (m >= 40) pass++; // unboxed inside the test
        System.out.println("Subjects passed (>= 40): " + pass + " of " + marks.length);
    }
    static int find(Integer[] a, Integer key) {
        for (int i = 0; i < a.length; i++)
```

```

        if (a[i].equals(key)) return i;           // equals, not ==, on objects
        return -1;
    }
}

```

10.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e10_wrappers

```

$ javac -d bin/e10_wrappers src/e10_wrappers/Main.java
$ java -cp bin/e10_wrappers Main
Enter the marks of three subjects (as lines):
Mathematics: 85
Physics: 78
Chemistry: 92
Total   : 255
Average : 85.0
Highest : 92 (Chemistry)
Subjects passed (>= 40): 3 of 3
$ java -cp bin/e10_wrappers Main
Enter the marks of three subjects (as lines):
Mathematics: 50
Physics: 45
Chemistry: 61
Total   : 156
Average : 52.0
Highest : 61 (Chemistry)
Subjects passed (>= 40): 3 of 3

```

10.4 Observations & Analysis

1. Case 1 (85, 78, 92): Total : 255 and Average : 85.0 — 255/3 exact; a hand check the marker can do during the demo.
2. Case 2 (50, 45, 61): total 156, average 52.0 — and Highest 61 (Chemistry) proved find() uses equals(), not ==, on boxed values.
3. “Subjects passed (>= 40): 3 of 3” printed in both sessions — the unboxed comparison inside the test, no intValue() visible in source.
4. The echoed lines sit exactly after their prompts in the transcript (Mathematics: 85 ...) — the pseudo-terminal capture records the bench session, guard-free input included.

10.5 Result

Three subjects’ string marks were converted through Integer wrappers, summed and averaged with compiler-inserted box/unbox cycles, and verified against hand arithmetic on two data sets.

10.6 Viva Voce Questions

1. Difference between Integer.valueOf(s) and Integer.parseInt(s)?
2. What does the compiler write behind total += m;?
3. Why does the average print 85.0, not 85?
4. If two marks were both 130, would m == highest inside find() work?

Model answers

Q1. Difference between Integer.valueOf(s) and Integer.parseInt(s)?

Ans. valueOf hands you the boxed object (may return a cached instance for −128...127); parseInt hands you the raw int. Exp. 10 needs the object, so valueOf — and it is also what Integer m = 85 means.

Q2. What does the compiler write behind total += m;?

Ans. total = Integer.valueOf(total.intValue() + m.intValue()) — one unbox, one add, one re-box, per round.

Q3. Why does the average print 85.0, not 85?

Ans. The numerator is cast to double before dividing, so the result is a Double whose toString keeps the decimal; integer / 3 would have truncated toward the floor.

Q4. If two marks were both 130, would `m == highest` inside `find()` work?

Ans. No — 130 is outside the cache, two objects, false identity. That is why `find()` compares with `equals()`; this is also the classic Module 5 viva trap, pre-empted in the source comment.

10.7 Exercises & Extensions

1. Read the three subject names too and store a double[] in parallel; print a formatted report card.
2. Switch the container to `ArrayList<Integer>` — one line for count-agnostic subjects.
3. Feed one non-numeric mark ("9x"): predict, then catch `NumberFormatException` with a re-prompt (reuse the Exp. 11 helper).
4. Count how many marks are above average using unboxed arithmetic twice — and say where each unbox lives.

Experiment 11 — Micro project — Student Marks Manager

AIM	Apply the course's OOP concepts to a small real-world console application, built and documented as a team of two to three students.
PROGRAMS	src/e11_project/
ENVIRONMENT & MATERIALS	OpenJDK 11 on Linux — compiled with javac 11, run with java 11; sessions captured from a real pseudo-terminal (echoed input exactly as typed). The IDE path (Eclipse/IntelliJ, most recent version) for the same sources is Appendix A.4.
COURSE OUTCOMES	CO6 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

11. Micro Project: A team of two to three students shall develop a project and submit a brief report for a specific task assigned by the course instructor based on the concepts covered in the syllabus.

11.1 Approach

One problem, every module. Person → Student is Module 3's inheritance with a super(name, phone) call; StudentRegistry is Module 2's encapsulation (a private list, no setters that matter); enum Grade plus the grade() ladder is Module 5 meeting Module 1; marks are pushed through Integer boxes into an ArrayList<Integer> (the autobox of Module 5 in the wild); and addMark(String) catches its own NumberFormatException, recording 0 with a warning instead of killing the session — Module 4 as a design opinion. The five-option menu keeps the whole app inside the vocabulary of Module 1's control statements.

The record-writing advice is equally concrete: the session below entered Aarav (85/92/88) and Diya (78/66/9x) — the malformed mark is in the transcript on purpose. Guards, toppler arithmetic and the found-record line are what the "Results & interpretation/analysis" rubric row pays for, and they are the lines a viva examiner asks to be walked through live. The IDE note belongs in the report: the same sources, compiled and debugged in the most recent Eclipse or IntelliJ (Appendix A.4), breakpoint on addMark's parse.

11.2 Program Code

The listings below are the files executed in §11.3: same bytes, same run — the transcript headers even print the file list handed to javac.

```
src/e11_project/Main.java · 116 lines, max width 93 cols

import java.util.Scanner;
import java.util.ArrayList;
/* Experiment 11 -- MICRO PROJECT: "Student Marks Manager", a console app
   for a batch of 2-3 students that gathers the course's OOP tools:
   inheritance (Person -> Student), constructors and super(...),
   encapsulation, enums for the grade, autoboxing inside ArrayList,
   dynamic dispatch through toString(). */

enum Grade { A, B, C, D, F }

class Person {
    private final String name, phone;
    Person(String name, String phone) { this.name = name; this.phone = phone; }
    String name() { return name; }
    public String toString() { return name + " <" + phone + ">"; }
}

class Student extends Person {
    private final String usn;
    private final ArrayList<Integer> marks = new ArrayList<>(); // boxes ints
    Student(String usn, String name, String phone) {
        super(name, phone); // parent constructor first
        this.usn = usn;
    }
    void addMark(String raw) { // parsing happens at the boundary
        try {
```

```

        marks.add(Integer.parseInt(raw.trim())); // autoboxes to Integer
    } catch (NumberFormatException e) {
        System.out.println(" bad number '" + raw + "' -- recorded as 0");
        marks.add(0);
    }
}

int total() {
    int t = 0;
    for (Integer m : marks) t += m; // unboxes each element
    return t;
}

double average() { return marks.isEmpty() ? 0 : total() / (double) marks.size(); }
Grade grade() {
    double a = average();
    if (a >= 90) return Grade.A;
    if (a >= 80) return Grade.B;
    if (a >= 70) return Grade.C;
    if (a >= 60) return Grade.D;
    return Grade.F;
}

String usn() { return usn; }
public String toString() {
    return String.format("%-12s %-10s marks=%-14s avg=%5.1f grade=%s",
        usn, name(), marks, average(), grade());
}
}

class StudentRegistry {
    private final ArrayList<Student> list = new ArrayList<>();
    void add(Student s) { list.add(s); }
    int size() { return list.size(); }
    Student find(String usn) {
        for (Student s : list)
            if (s.usn().equalsIgnoreCase(usn)) return s;
        return null;
    }
    Student topper() {
        Student best = null;
        for (Student s : list)
            if (best == null || s.average() > best.average()) best = s;
        return best;
    }
    void printAll() {
        if (list.isEmpty()) { System.out.println(" (no students yet)"); return; }
        for (Student s : list) System.out.println(" " + s); // dispatches to toString
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StudentRegistry reg = new StudentRegistry();
        while (true) {
            System.out.println();
            System.out.println("1.Add student 2.Show list 3.Topper 4.Search 5.Exit");
            System.out.printf("Choice: ");
            switch (sc.nextInt()) {
                case 1: {
                    sc.nextLine();
                    System.out.printf("USN: "); String usn = sc.nextLine();
                    System.out.printf("Name: "); String name = sc.nextLine();
                    System.out.printf("Phone: "); String ph = sc.nextLine();
                    Student s = new Student(usn, name, ph);
                    for (int i = 1; i <= 3; i++) {
                        System.out.printf("Mark for subject " + i + ": ");
                        s.addMark(sc.nextLine());
                    }
                    reg.add(s);
                    System.out.println("Added: " + s);
                    break;
                }
                case 2: reg.printAll(); break;
                case 3: {

```

```

        Student t = reg.topper();
        System.out.println(t == null ? " (no students yet)" : " Topper: " + t);
        break;
    }
    case 4: {
        sc.nextLine();
        System.out.printf("USN to find: ");
        Student s = reg.find(sc.nextLine().trim());
        System.out.println(s == null ? " not found" : " found: " + s);
        break;
    }
    case 5: System.out.println("bye"); return;
    default: System.out.println("invalid choice");
}
}
}
}
}

```

11.3 Compilation & Execution (Captured)

Captured sessions follow: the compile line, then each run. Typed input appears on the same line as its prompt, echoed by the terminal exactly as at the bench. Guard runs (bad input, empty stack, unknown code) are included on purpose — they are part of the evidence the rubrics pay for.

e11_project

```

$ javac -d bin/e11_project src/e11_project/Main.java
$ java -cp bin/e11_project Main

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 1
USN: 1AB23CS001
Name: Aarav
Phone: 9810011223
Mark for subject 1: 85
Mark for subject 2: 92
Mark for subject 3: 88
Added: 1AB23CS001 Aarav marks=[85, 92, 88] avg= 88.3 grade=B

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 1
USN: 1AB23CS014
Name: Diya
Phone: 9900223344
Mark for subject 1: 78
Mark for subject 2: 66
Mark for subject 3: 9x
bad number '9x' -- recorded as 0
Added: 1AB23CS014 Diya marks=[78, 66, 0] avg= 48.0 grade=F

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 2
1AB23CS001 Aarav marks=[85, 92, 88] avg= 88.3 grade=B
1AB23CS014 Diya marks=[78, 66, 0] avg= 48.0 grade=F

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 3
Topper: 1AB23CS001 Aarav marks=[85, 92, 88] avg= 88.3 grade=B

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 4
USN to find: 1AB23CS014
found: 1AB23CS014 Diya marks=[78, 66, 0] avg= 48.0 grade=F

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 5
bye
$ java -cp bin/e11_project Main

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 2
(no students yet)

```

```

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 3
(no students yet)

1.Add student 2.Show list 3.Topper 4.Search 5.Exit
Choice: 5
bye

```

11.4 Observations & Analysis

1. Added row printed on the spot: 1AB23CS001 Aarav marks=[85, 92, 88] avg= 88.3 grade=B — 265/3 = 88.333→88.3, band B (≥80).
2. Diya's "9x" produced "bad number '9x' — recorded as 0" and the row avg= 48.0 grade=F — 144/3 = 48.0; the app survived a bad cell, which is the exception story in product.
3. Topper line named Aarav again after both records existed — a scan over the registry, comparing averaged doubles through wrapper-free Student methods.
4. Search by "1AB23CS014" found Diya; case 2 of the second session shows the honest empties: "(no students yet)" for topper, list empty — the app behaves before data arrives.
5. Every menu cycle reprints the banner (case 1 → case 2 → 3 → 4 → 5 in the log): the transcript doubles as a usability demo for the viva.

11.5 Result

A console Student Marks Manager was delivered for a 2-3 student batch, exercising classes, inheritance, enum grades, wrapper-autoboxed lists, exception-tolerant input and a menu loop, verified across two sessions including a malformed-mark recovery.

11.6 Viva Voce Questions

1. Where exactly does inheritance earn its keep in this app (name the line)?
2. Why does addMark swallow its exception instead of rethrowing?
3. Which module's vocabulary is the Grade ladder?
4. For the report's IDE section, which debugger action do you actually demonstrate?

Model answers

Q1. Where exactly does inheritance earn its keep in this app (name the line)?

Ans. Person holds name/phone and Student calls super(name, phone) — any future Teacher could reuse Person untouched; plus toString chains Student's format over Person's name().

Q2. Why does addMark swallow its exception instead of rethrowing?

Ans. A menu app that dies on one bad keystroke fails the UX test; the registry must stay usable. A library would instead wrap and rethrow — the difference is context, and the viva is asking whether you know which you are in.

Q3. Which module's vocabulary is the Grade ladder?

Ans. Module 1 — the if/else chain returning enum constants; Module 5 only supplied the type.

Q4. For the report's IDE section, which debugger action do you actually demonstrate?

Ans. A breakpoint on addMark's parse line, one run feeding "9x", stepping the catch — the failure path shown, not asserted.

11.7 Exercises & Extensions

1. Swap the menu for a file-loaded batch (one CSV) — parse with String.split, same guards.
2. Add a delete-by-USN and a persist-to-file exit; now the app has state across sessions.
3. Introduce an interface Reportable (println()) and make Student and StudentRegistry each implement it.
4. Give every student an attendance percentage; grade() becomes weighted — keep the enum untouched and prove why.

P A R T



Appendices

The reference plates for the whole book: the language sheet and the two ways to run it, the record format with the university's own rubric tables and the 420→15 arithmetic, a consolidated viva bank, and the resources with the self-study plan.

IN THIS PART

- Appendix A — Java Quick Reference
- Appendix B — Record Format, Rubrics & Assessment Arithmetic
- Appendix C — Viva Voce Bank (30 questions)
- Appendix D — Resources, TW/SL Plan & Reproducing This Book
- About the Author

A.1 The five lexical elements, one line each

Whitespace	Blanks/newlines separate tokens; only inside a string literal do they count.
Identifiers	Letters, digits, <code>_</code> , <code>\$</code> ; cannot start with a digit; case-sensitive; no keyword.
Literals	<code>42</code> <code>0x2A</code> <code>0b1010</code> <code>052</code> <code>100_000L</code> <code>3.14</code> <code>6.02e23</code> <code>'A'</code> <code>'\u0041'</code> <code>"s"</code> <code>true</code> .
Comments	<code>// line</code> , <code>/* block */</code> , <code>/** javadoc */</code> — no nesting of block comments.
Separators	<code>() { } [] ; ,</code> plus every operator symbol.

Keywords: 50 reserved words in Java 11 (the list the reference book prints in Ch. 2); `const` and `goto` are reserved-but-unused; `true/false/null` are literals, not keywords. Do not memorise the list — memorise the error the compiler throws when you try: Appendix A.5.

A.2 Types and operators, with captured bounds

Primitive	Size	Range — as printed by <code>t1_types</code>
byte	1 B	−128 ... 127
short	2 B	−32 768 ... 32 767
int	4 B	−2 147 483 648 ... 2 147 483 647
long	8 B	$\pm 9.22 \times 10^{18}$
char	2 B	unsigned Unicode 0 ... 65 535
float / double	4 / 8 B	$\approx 7 / \approx 15$ –16 significant digits
boolean	—	true / false only

Precedence (high → low)	Operators
unary	<code>++ -- ! ~ + - (cast)</code>
multiplicative	<code>* / %</code>
additive	<code>+ -</code>
relational	<code>< > <= >= instanceof</code>
equality	<code>== !=</code>
logical	<code>&& </code> (each short-circuits)
ternary	<code>? :</code>
assignment	<code>= += -= *= /= %=</code> (right-associative)

When in doubt the syllabus's own remedy is one keystroke: parentheses. Every operand smaller than `int` is promoted before any of this runs.

A.3 Command-line workflow — the exact plate every transcript in this book was printed from

```
javac -d bin/myprog src/myprog/Main.java           # compile into bin/
java -cp bin/myprog Main                          # run: classpath = dir ABOVE the package tree
javac -d bin/e06_pkg src/e06_pkg/mypack/Calculator.java src/e06_pkg/mypack/user/Demo.java
java -cp bin/e06_pkg mypack.user.Demo              # main class stays package-qualified
javac src/x/*.java                                 # legal for one default-package file, then:
java -cp src/x Main                                # classes found next to sources
```

`-d` writes `.class` files into a package-shaped tree; without it the JVM and your folder quietly disagree (A.5's most common runtime error).

One public type per file; file name = type name; package `a.b`; = directory `a/b/`.

Scanner input needs no special flag; run interactively to see the prompt-and-echo pattern the record shows. Compile with `-Xlint:all` to see the two warnings this book's `System.gc()` mention provokes elsewhere.

A.4 The IDE route (Eclipse / IntelliJ IDEA, most recent version)

The practical note asks for the most recent IDE; the mapping below is mechanical, so an IDE-built record prints the same transcripts. Install the current JDK (17/21/25 any works for every source in this book) and either IntelliJ IDEA Community or Eclipse IDE for Java Developers; both consume the sources as-is (the module demos are single files; Experiments 6 is a two-package tree — import the src/e06_pkg folder as the project root so mypack/ sits under src).

Book step	IntelliJ IDEA	Eclipse
New program	New Project → Java, JDK picked under Project Structure	New Java Project → JRE 11+ → class in src
Compile	Build (⌘F9) — output to out/production/<proj>	Automatic build — bin/
Run	Run ► (Shift+F10) in the Run tool window console	Run → Java Application (Ctrl+F11 → Console view)
Interactive stdin	untick “Emulate terminal” for the paste-flow, or tick Terminal for the typed feel	Console accepts typing directly
Debugger (Exp. 11’s viva line)	breakpoint in addMark → Debug □ → F8 step / F7 into	breakpoint → Debug → F6/F5, Variables view
Transcript for the record	copy from the Run window; header = the console’s first line	same; Console → Select All → Copy

The IDE adds nothing the language requires — every transcript here was produced from the shell. What the IDE adds is the debugging half of the demonstration rubric row (“Program debugging and testing with suitable tools”), which is why the syllabus names it.

A.5 Six compiler messages, captured from javac 11

```
$ (cd docs/snips && for each file below) javac <file>.java
```

```
$ javac S1_byte.java S1_byte.java:2: error: incompatible types: possible lossy conversion from int to byte
byte p = 10, q = 20; byte r = p + q; System.out.println(r); } } ^ 1 error
$ javac S2_semi.java S2_semi.java:2: error: ';' expected
int x = 5 ^ 1 error
$ javac S3_priv.java S3_priv.java:3: error: code has private access in Vault
System.out.println(new Vault().code); } } ^ 1 error
$ javac S4_final.java S4_final.java:2: error: cannot assign a value to final variable MAX
final int MAX = 5; MAX = 6; System.out.println(MAX); } } ^ 1 error
$ javac S5_unreach.java S5_unreach.java:3: error: unreachable statement
System.out.println("after"); } } ^ 1 error
$ javac S6_return.java S6_return.java:1: error: missing return statement
public class S6_return { static int pick(boolean t) { if (t) return 1; } } ^ 1 error
```

Lines 1 and 6 are the two Module-1/2 exam questions hiding as errors: promotion refuses a silent narrowing; the compiler counts exit paths for every return type.

Lines 3–4 are the access-control and final sentences of Module 2/3, printed by the compiler before you can ever make the runtime mistake.

Line 5 is why return inside a loop is sometimes cleaner than code after a while(true).

Appendix B — Record Format, Rubrics & Assessment Arithmetic

B.1 The record skeleton (each experiment, six sections to match the six rubric rows)

#	Section	What the row on the rubric sheet pays for
1	Aim & problem statement (verbatim)	Fundamental Knowledge — quoting the question word-for-word is the cheapest 5 marks in the sheet.
2	Approach / design notes (why this class split, alternatives rejected)	Design of program — “discusses multiple designs and selects the best with justification” is the Superior column; Part B’s Approach sections model one.
3	Program code (complete, listing every file)	Implementation using Java — compilable, commented, OOP-visible.
4	Compilation & execution transcript (javac line + java run(s))	Debugging & testing — the transcript proves it compiles; guard runs prove testing.
5	Observations / results (numbered, each citing a transcript line)	Results & interpretation — multiple test cases get Superior only if they are analysed.

#	Section	What the row on the rubric sheet pays for
6	Conclusion + viva notes (the chapter's Q/A pairs you can say aloud)	Demonstration & documentation, and the test-rubric's Communication row.

B.2 The arithmetic

Each experiment (1 to 10) will be evaluated for 30 marks (total 300), and the micro-project for 120 marks. Total marks (420) shall be scaled down to 15. Make use of the most recent version of the IDE tool for program development. Concretely: ten experiment reports scored out of 30 each (six rubric criteria × 5) plus the micro project out of 120, all out of 420 — a student averaging “good” on everything earns $420 \times 4/5 = 336$, and $336/420 \times 15 = 12.0 / 15$. The lab test runs 3 hours for 50 marks (one experiment, micro project excluded), scaled ÷5 for 10; report-15 + test-10 = practical CIE 25; the two IA tests average to theory CIE 25; CIE 50 + SEE 50 = 100 with the 10/25 + 10/25 eligibility gates and the 18/50 SEE gate.

B.3 CIE rubric — laboratory report (as printed in the syllabus PDF)

Criteria / Scale	Superior (5)	Good (4)	Fair (3)	Needs Improvement (2)	Marginal (1)
Fundamental Knowledge: Understanding the problem statement [CO1] [PO1, PO2]	The student has in-depth knowledge of the topics related to the problem. Student is able to completely understand the problem definition.	Student has good knowledge of some of the topics related to problem. Student is able to understand the problem definition.	Student is capable of narrating the answer but not capable to show in-depth knowledge and the problem definition.	Student has not understood the concepts partially. Student is able to partially understand the problem definition.	Student has not understood the concepts and the problem definition clearly.
Design of program [CO2-6] [PO2, PO3]	Capable of discussing multiple designs and selecting the best with justification.	Discusses multiple designs but lacks proper selection criteria.	Discusses a single design with merits and demerits.	Explains only the basic design.	Unable to explain the design properly.
Implementation using Java [CO2-6] [PO3, PO8, PO5]	Implements optimal solution using suitable Java constructs and OOP features.	Correct implementation with good explanation.	Correct implementation with partial explanation.	Implementation contains errors.	Unable to implement the solution completely.
Program debugging and testing with suitable tools [CO2-6] [PO5, PO8]	Compiles, debugs, and tests without errors; demonstrates full understanding.	Corrects errors with good understanding.	Corrects errors with partial understanding.	Corrects errors with assistance.	Unable to debug and test independently.
Results & interpretation/analysis [CO1-6] [PO4, PO5]	Executes for multiple test cases and performs detailed analysis.	Executes correctly for all test cases.	Executes for a few test cases and analyzes results.	Executes but cannot analyze results.	Unable to verify correctness of results properly.
Demonstration and documentation [CO1-6] [PO8, PO9, PO11]	Demonstration and lab record is well-organized, with clear sections. The record is well structured with suitable formatting.	Demonstration and lab record is organized, with clear sections, but some sections are not well-defined. The record is structured with formatting.	Demonstration and lab record lacks clear organization or structure. Some sections are unclear or incomplete. The record is partially	Demonstration and lab record is poorly organized, with missing or unclear sections. The record is not properly structured with suitable formatting.	Demonstration and lab record is poorly organized, with missing sections. Record not submitted on time. The record is not structured with

Criteria / Scale	Superior (5)	Good (4)	Fair (3)	Needs Improvement (2)	Marginal (1)
			structured with formatting.		minimum formatting.

B.4 CIE rubric — practical test

Criteria / Scale (weights sum to 10 before scaling)	Superior (5)	Good (4)	Fair (3)	Needs Improvement (2)	Marginal (1)
Fundamental Knowledge (2) [CO1] [PO1]	The student has well depth knowledge of the topics related to the problem & course.	Student has good knowledge of some of the topics related to problem & course.	Student has average knowledge of some of the topics related to problem & course.	Student is capable of narrating the answer but not capable to show in-depth knowledge.	Student has not understood the concepts clearly.
Understanding of problem definition (1) [CO2-6] [PO2]	Student is able to completely understand the problem definition.	Student is able to understand the problem definition but not clearly.	Student has a basic understanding of the problem definition that is partial or superficial.	Student is able to show minimal or unclear understanding of the problem definition.	Student is not able to understand the problem definition clearly.
Design and Implementation (3) [CO2-6] [PO3, PO5]	Student is capable of design and implementing with best suitable Java construct for the given problem definition.	Student is capable of design and implementing with some construct for the given problem definition.	Student is capable of design and implementing the core part of the construct for the given problem definition.	Student is partially capable of design and implementing with some algorithm for the given problem definition.	Student is not capable of design and implementing completely.
Result & Analysis (2) [CO2-6] [PO4, PO5]	Student is able to run the program on various data inputs and compare the result with proper inference.	Student will be able to run the program on various data inputs and fair knowledge in comparing the result with proper inference.	Student will be able to run the code for few data/datasets and analyze the output.	Student will be able to run the code for few data inputs but not analyze the output.	Student will be not able to run the program and not able to analyze the result.
Communication (Viva voce) (2) [CO1-6] [PO8, PO9]	Good verbal & nonverbal communication skills with precise and correct terminologies/ answers.	Good verbal communication skills with precise and correct terminologies/ answers.	Average communication but with precise and correct terminologies/ answers.	Average communication but with imprecise and incorrect terminologies/ answers.	Poor communication (minimal interaction/ answers).

Appendix C — Viva Voce Bank (30)

Answers are phrased for speaking; each cites the chapter evidence. Numbers in brackets are transcript values examiners may ask you to reproduce.

#	Question	Answer, short
1	Why does Java need a class for even one print line?	The language has no top-level statements; main is a static method the JVM looks up by signature. (M1, t1_overview)
2	JDK vs JRE vs JVM?	JVM executes bytecode; JRE = JVM + libraries to run; JDK = JRE + javac/tools to build. Book ran OpenJDK 11. (App A.3)
3	byte p+q in int — why?	Automatic type promotion: arithmetic below int width happens at int; assign back and you get lossy-conversion. (A.5 first error)
4	What does (int)9.65 give?	9 — casts truncate toward zero, never round. (t1_types: 9)
5	MAX_VALUE + 1?	-2147483648 as int; 2147483648 if one side is long. (t1_types)

#	Question	Answer, short
6	switch vs if ladder?	switch: one value against constant labels; if: ranges/relational. Grade “≥80” is a ladder job. (t1_control)
7	for-each vs for(var i...)?	for-each: elements, no index; var: compile-time-inferred index loop. Both are Module-1 iteration lines. (t1_control)
8	new Box() — three jobs?	Allocate, run a constructor, return the reference. (M2)
9	What is copied by Box d = b;?	Only the reference; set through d and b sees 1000.0. (t2_classes)
10	this() vs this.x?	Constructor call between overloads vs. field disambiguation — same keyword, two duties. (Box)
11	Overload or override: area(double) in two classes?	Same signature across a hierarchy = override (run-time); within one class with different args = overload (compile-time). (M2/M3)
12	Why does factorial(10)=3628800 not crash?	Base case 1 stops ten frames deep; ints hold the result. (t2_methods)
13	static int calls — where does it live?	On the class; all objects share it — the demo counted 3 across three calls. (t2_methods)
14	Constructor order in A→B→C?	Parent first: A(), B(), C(). (t3_inherit)
15	What makes speak() polymorphic?	Parent reference + overridden body + run-time lookup of the object’s class. (Rex barks)
16	instanceof — when is it honest?	Before a downcast, exactly as the fetch line does. (t3_inherit)
17	Abstract vs interface in one breath?	Abstract class: partial implementation + state; interface: contract (+default/static machinery), multi-implementable. (t3_iface)
18	What crosses a package border?	public members only; the bridge class exposed the 99. (t3_pkg)
19	Why java -cp bin/x mypack.user.Demo?	-cp points above the tree; launch needs the FQCN. (Exp 6)
20	Checked vs unchecked?	Checked: compiler demands catch/declare (IOException); unchecked: RuntimeException/Error free-fall. (Exp 7)
21	throw vs throws?	Statement that raises vs signature that promises. (t4_except)
22	Order of two catches?	Specific first; general-before-specific is a compile error (already caught by the sub-block design). (t4_except)
23	finally skips what?	Nothing except JVM death (System.exit); ran on both 10/2 and 10/0. (t4_except)
24	Chained exception?	Wrap cause in super(msg, cause); getCause() unwraps “ledger read failed...”. (t4_except)
25	start() vs run()?	start schedules a new thread; run() called directly is just a method. (Exp 8)
26	join() contract?	Caller waits for that thread’s death; children printed after it. (t4_threads)
27	Priority semantics?	Hints to the scheduler: 10 vs 1 changes contention, not ordering. (t4_threads)
28	Enum identity?	One instance per constant; == is safe; valueOf is case-exact — hence trim().toUpperCase(). (Exp 9)
29	Integer cache range?	-128...127: 127==127 true, 128==128 false; equals never lies. (t5_enum)
30	What does autoboxing insert into total += m?	intValue, int add, valueOf back. (Exp 10)

Appendix D — Resources, TW/SL Plan & Reproducing This Book

D.1 Books (as the syllabus prescribes)

Textbook: Herbert Schildt and Danny Coward, Java: The Complete Reference, 13th Edition, McGraw-Hill, 2024.

References:

Cay S. Horstmann, Core Java Volume I - Fundamentals, 14th Edition, Oracle Press, 2026.

E. Balagurusamy, Programming with Java, 7th Edition, McGraw-Hill Education, 2024.

D.2 e-Resources (official links)

[Official Java documentation \(Oracle\)](https://docs.oracle.com/javase/tutorial/) — <https://docs.oracle.com/javase/tutorial/>

[Modern Java learning resources \(Dev Java\)](https://dev.java/learn/) — <https://dev.java/learn/>

[Java Tutorial](https://www.w3schools.com/java/) — <https://www.w3schools.com/java/>

[Programming examples and practice \(GeeksforGeeks\)](https://www.geeksforgeeks.org/java/java/) — <https://www.geeksforgeeks.org/java/java/>

[Infosys Springboard Course on Java Programming Fundamentals](https://infyspringboard.onwingspan.com/web/en/login) — <https://infyspringboard.onwingspan.com/web/en/login>

[HackerRank platform](https://www.hackerrank.com/domains/java) — <https://www.hackerrank.com/domains/java>

[Structured video lectures \(NPTEL\)](https://nptel.ac.in/courses/106105191/) — <https://nptel.ac.in/courses/106105191/>

D.3 Teaching-learning methods, TW and SL (verbatim rows)

Innovative delivery methods listed by the syllabus: Flipped Classroom. · Problem-Based Learning · ICT Enabled Teaching · Programming Assignments · Tool-Based Learning using Eclipse/IntelliJ IDEA.

Type	Activity (as printed)	Hours
TW	Installation and use of IDE for Java. Writing simple Java programs to understand Java program development and execution model. — 04 hours/semester	04
SL-1	Infosys Springboard course: Java Programming Fundamentals	36
SL-2	HackerRank Java problems (https://www.hackerrank.com/domains/java)	10

D.4 Reproducing every transcript in this book

All listings live under `src/<stem>/`; the harness (`collect.py`) compiles each stem into `build/bin/<stem>`, drives the interactive runs through a pseudo-terminal with the scripted inputs, and writes out `<stem>.txt` — the files embedded verbatim in Parts A and B. One command rebuilds the whole evidence folder:

```
cd java-course && python3 collect.py      # 23 stems, ~40 s
python3 build.py                        # render PDF (two passes + footers)
python3 qa.py                          # the checks behind the colophon
```

On a machine with a newer JDK the outputs are byte-identical for these sources (no preview features, no locale-dependent lines except the exception texts); the thread demos' state-sampler line may report a different transient state — say so in the record if it does; that sentence is the demonstration.

About the Author

Dr. Lokesh M R is Professor in the Department of Information Science & Engineering at A J Institute of Engineering and Technology (AJIET), Mangaluru, teaching first-year computing under the VTU 2025 scheme, including this integrated Java course. His teaching practice centres on documentation that can be defended orally — the reason every chapter in this book pairs a worked program with a captured transcript and a viva trail. He is also the author of the companion volumes in the same series and format: C Programming Lab, Python Programming (IPCC) and Probability, Distributions & Statistics.

Corrections, suggestions and classroom reports are welcome at the department office. First Edition, September 2026 — programs verified against OpenJDK 11 on the publication date; the book's sources are Java 8-11 clean, so any current JDK and IDE compiles them unchanged.